



Land use and land cover classification on high-resolution UAV images for heavy rainfall hazard maps using deep neural networks

Dennis Sprute^a,^{*} Hanne Hendrickx^b, Pedro Zamboni^b,
Muhtasimul Islam Rushdi^c, Florian Brodrecht^c, Anette Eltner^b, Holger Flatt^a

^a Fraunhofer IOSB, Industrial Automation Branch (IOSB-INA), 32657 Lemgo, Germany

^b Institute of Photogrammetry and Remote Sensing, Dresden University of Technology, 01062 Dresden, Germany

^c SPEKTER GmbH, 91074 Herzogenaurach, Germany

ARTICLE INFO

Keywords:

Land use/cover classification
UAV
RGB images
Heavy rainfall hazard maps
Semantic segmentation
Deep neural networks

ABSTRACT

The increasing frequency and intensity of extreme weather events due to climate change highlight the need for accurate hazard mapping for heavy rainfall. Land use and land cover (LULC) maps, along with digital elevation models (DEMs) and meteorological data, are essential to construct these hazard maps. For this purpose, LULC maps must be of high resolution to capture small hydrodynamically relevant structures and up-to-date to reflect constant changes driven by human activities. Uncrewed aerial vehicles (UAVs) are ideal for acquiring these high-resolution images because of their cost effectiveness, flexibility, and detail. However, no publicly available datasets and approaches currently provide urban LULC maps at this level of detail (<5 cm) while considering classes relevant for constructing heavy rainfall hazard maps. Therefore, this article presents a novel approach for LULC classification using high-resolution UAV imagery to enhance the generation of heavy rainfall hazard maps. We develop a comprehensive processing pipeline that includes UAV data collection, orthophoto creation, and LULC classification using advanced deep learning architectures. Our method tackles challenges such as differing between 22 LULC classes with objects of varying sizes and managing imbalanced datasets. Additionally, we create a high-resolution UAV image dataset with more than 12K annotated images. A comprehensive evaluation of 29 deep learning architectures reveals that CFNet with an EfficientNetB4 backbone achieves the best performance, with an accuracy of 0.911 and a Mean Intersection over Union (IoU) of 0.738. The results demonstrate that our approach effectively classifies hydrodynamically relevant structures, such as different roof types, street materials, and drainage facilities. This work establishes a solid foundation for future research and practical applications in the automated generation of accurate heavy rainfall hazard maps, ultimately aiming to improve risk management strategies in the context of climate change.

1. Introduction

The rising frequency and intensity of extreme weather events resulting from climate change present considerable challenges to rural and urban areas. Heavy rainfall is a particularly concerning phenomenon among these events, as it can lead to severe flooding, damage to infrastructure, and substantial economic losses (Wang et al., 2024; Leal et al., 2019; Jongman et al., 2014).

^{*} Corresponding author.

E-mail address: dennis.sprute@iosb-ina.fraunhofer.de (D. Sprute).

<https://doi.org/10.1016/j.rsase.2025.101609>

Received 29 January 2025; Received in revised form 25 April 2025; Accepted 23 May 2025

Available online 9 June 2025

2352-9385/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Evidence suggests that large flood events, which can pose a danger to people and infrastructure, are increasing (Wasko et al., 2021). For example, the highest number of damage incidents in Germany has been caused by short-duration and intense rainfall events, resulting in 1.3 million cases of damage to residential buildings and a total cost of 7.4 billion euros from 2002 to 2018 (GDV and DWD, 2019). Therefore, it is essential to implement effective risk management strategies to minimize the adverse impacts of such events. A key component of these strategies is the development of high-resolution hazard maps, which serve as a crucial tool to identify and mitigate risks (Kimura et al., 2023).

Heavy rainfall hazard maps illustrate the potential for surface flooding and the movement of runoff water during heavy rainfall events. These maps demonstrate how water flows over terrain depressions and channels towards water bodies. The utility of these maps has been demonstrated to facilitate informed decision-making on mitigation of flood damage and protection of public safety (Rangari et al., 2019). Governmental bodies use such maps to delineate flood-prone regions and identify potential hazards (de Moel et al., 2009). Thus, they play an important role in urban planning, disaster management, and emergency preparedness.

These maps are constructed utilizing digital elevation models (DEMs), which are usually obtained via airborne laser scanning (ALS). The essential meteorological input for creating the hazard map is precipitation, combined with other indicators related to specific surface and soil properties, such as soil and land cover/use maps (LUBW, 2016). The process of heavy rainfall hazard mapping mainly consists of three steps: (1) mesh generation, (2) mesh parametrization and (3) flood simulation. The first step creates a mesh structure from the DEM and segments the terrain into grid cells to illustrate various land types and hydraulic elements, such as curbs, ditches, and culverts. Subsequently, mesh parametrization is performed, which assigns physical properties (e.g. roughness, runoff curve number (CN) values) to each cell in the mesh. For this purpose, land use/land cover (LULC) maps are a valuable resource as the distinctive hydraulic and hydrological characteristics exhibited by each land cover type exert a significant influence on the rainfall-runoff generation process. For instance, permeable surfaces (such as bare soil and grasslands) can facilitate greater water infiltration and reduce the generation of runoff. In contrast, impermeable areas (e.g. paved streets and sidewalks) impede water infiltration and enhance runoff generation. Finally, flood simulation and hazard mapping is facilitated by using 1D and 2D hydrological and hydrodynamical models, such as MIKE and HEC-RAS (Moghim et al., 2023; Thrysoe et al., 2021; Yalcin, 2020).

Currently, this process of hazard mapping is largely manual, in particular mesh generation and mesh parametrization. It requires extensive post-processing to ensure the fidelity of the structures in the mesh. Small but hydraulically significant structures, like inlets and manholes, are often not captured by the resolution of existing DEM, which requires labor-intensive manual input and field verification. Moreover, mesh parametrization requires detailed and up-to-date classification of land types. However, these information are often not available affecting the precision of flood simulations. Such manual processes increase both the time and cost of producing accurate hazard maps.

Therefore, high-resolution and up-to-date LULC classification could benefit the automation of this hazard mapping process. Several LULC datasets have been developed, ranging from coarse resolution (100 m to 1 km) to medium resolution (10 m to 100 m) and high-resolution products (1 m to 10 m) (Li et al., 2024). However, the heterogeneity and complexity of urban areas make satellite platforms such as Landsat and Sentinel-2 unsuitable for high-detail monitoring, where small urban components are split between pixels, resulting in a mixed pixel effect that can lead to incorrect borders or even the complete disappearance of these objects (He et al., 2023). Moreover, cities are in constant change, therefore, LULC maps need to be constantly updated to ensure accurate hazard maps. In this context, Uncrewed Aerial Vehicles (UAVs) offer a relatively low-cost alternative for urban LULC mapping, producing high-resolution maps. These platforms are particularly valuable due to their flexibility, enabling rapid and efficient data collection, especially in newly developed areas. Nevertheless, there is a need for automatic methods to handle the amount of data and extract valuable information.

Deep learning, especially in the form of Convolutional Neural Networks (CNNs), has emerged as a state-of-the-art method for LULC mapping (Dou et al., 2024; Jia et al., 2024; Zhang et al., 2019a). For urban scenes, most UAV benchmark datasets, such as the AeroScapes (Nigam et al., 2018), UAVid (Lyu et al., 2020) and VDD (Cai et al., 2023) datasets, focus on more generic classes like roads, buildings, low and high vegetation. In the same way, most part of the studies on urban land cover combining UAV images and deep learning focus on the same set of classes (Yi et al., 2023; Wang et al., 2022; Girisha et al., 2019), therefore, not using classes that are important during the construction of heavy rainfall hazard maps. To the best of our knowledge, no publicly available dataset provides urban LULC maps at this level of resolution (<5 cm) and considers classes relevant for heavy rainfall hazard mapping. In this regard, datasets are needed that can be used to facilitate the creation of these hazard maps and to investigate the most appropriate models and training procedures.

Therefore, the objective of this work is the automatic LULC classification using high-resolution UAV image data to facilitate the generation of heavy rainfall hazard maps, in particular the mesh generation and mesh parametrization. Among the heterogeneity and complexity of urban areas, this task presents several challenges, such as the large number of relevant objects that need to be classified and the wide variety in object sizes, ranging from small objects such as manholes and inlets to large structures like buildings and streets. Additionally, the issue of imbalanced data, common in real-world scenarios, must be addressed.

Such a UAV-based LULC classification method will be a solid basis for automating parts of the mesh generation and mesh parametrization steps in hazard mapping by addressing the inefficiencies of the manual process. In particular, relevant LULC structures, not present in the ALS-based DEM due to an insufficient resolution or outdated data, can be integrated into meshes for flood simulation. This would enable more accurate hazard maps and could eliminate the need for labor-intensive field verifications. Additionally, physical properties of LULC structures can be automatically assigned to mesh cells, which is not possible today due to lack of high-resolution and up-to-date LULC maps. This would be a significant step towards improving the speed of creation and accuracy of heavy rainfall hazard maps.

To achieve this objective, we contribute the following points which collectively establish a robust foundation for future research and practical implementations in this field:

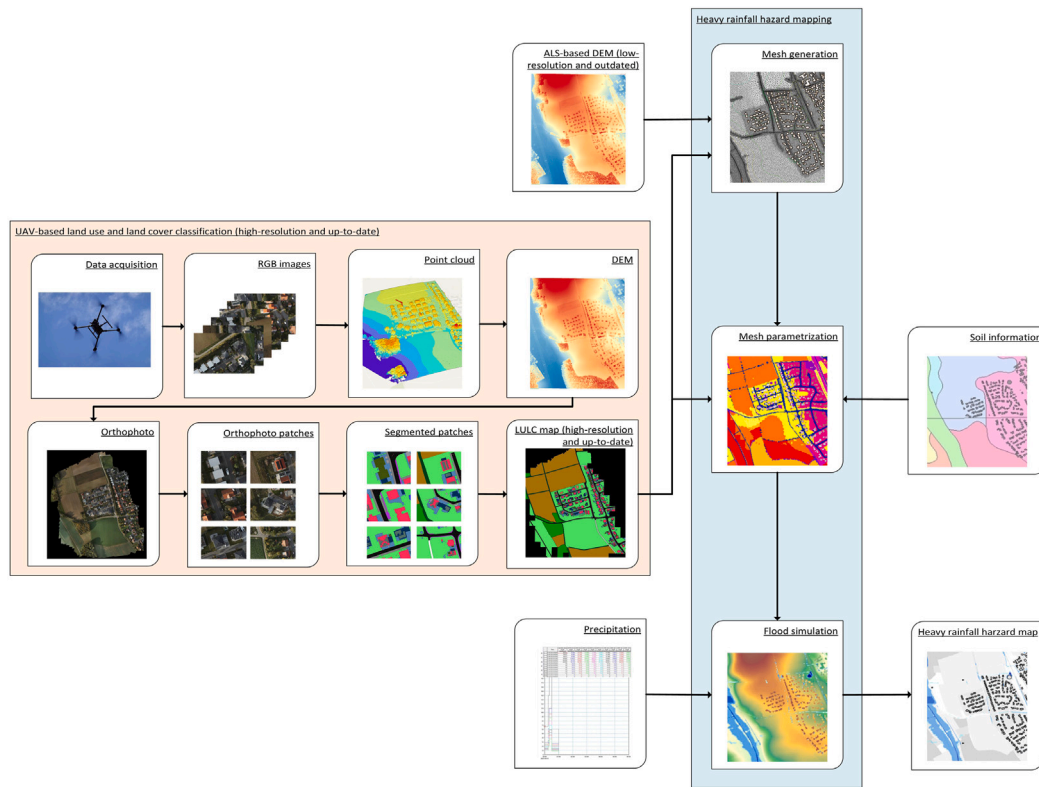


Fig. 1. Processing pipeline of the proposed UAV-based LULC classification system in the context of heavy rainfall hazard mapping. The blue box encompasses the steps of the current approach of hazard mapping consisting of the three steps mesh generation, mesh parametrization and flood simulation with its input data, i.e. a low-resolution and outdated ALS-based DEM, soil information and precipitation. The result of this process is the heavy rainfall hazard map. To automate parts of this process, we propose a UAV-based LULC classification method as shown in the orange box, which delivers high-resolution and up-to-date LULC maps.

1. A **classification system** of objects and structures relevant for the creation of heavy rainfall hazard maps
2. A novel **annotated high-resolution UAV image dataset** encompassing a wide variety of relevant objects and structures
3. A **comprehensive evaluation** of various state-of-the-art deep neural network architectures for semantic segmentation on the novel dataset
4. A **deep neural network-based method** specifically designed for LULC classification, with a focus on applications in heavy rainfall hazard mapping

The remainder of this article is structured as follows: In Section 2, we describe the materials and methods used in this study, including the proposed classification system of relevant objects and structures, the data collection and preprocessing, and the neural network architectures and training process. Section 3 presents the results of our experiments, offering quantitative and qualitative evaluations of the different models and a comparison with similar works. In Section 4, we discuss the implications of our findings, focusing on the impact of different backbone models, the effect of custom class weights, the role of different neural network architectures, limitations of our approach and the relevance of the results for automating the heavy rainfall hazard mapping process. Finally, Section 5 concludes the article and suggests potential directions for future research.

2. Materials and methods

To address the objective of an automatic UAV-based LULC classification method and facilitate the construction of heavy rainfall hazard maps, we propose the following processing pipeline, as illustrated in Fig. 1.

This method begins with data acquisition using a UAV equipped with an RGB camera. The UAV captures high-resolution color images of the environment annotated with geocoordinates. The high resolution of the images is necessary to take into account small objects in the environment. These images are then processed to generate a 3D point cloud, which is subsequently converted into a DEM and an orthophoto, serving as input for the LULC classification. Since each pixel in the orthophoto needs to be classified according to an object class, this task is formulated as a semantic segmentation problem from the field of computer vision (Garcia-Garcia et al., 2018). Unlike image classification, semantic segmentation involves pixel-level classification, where each pixel is

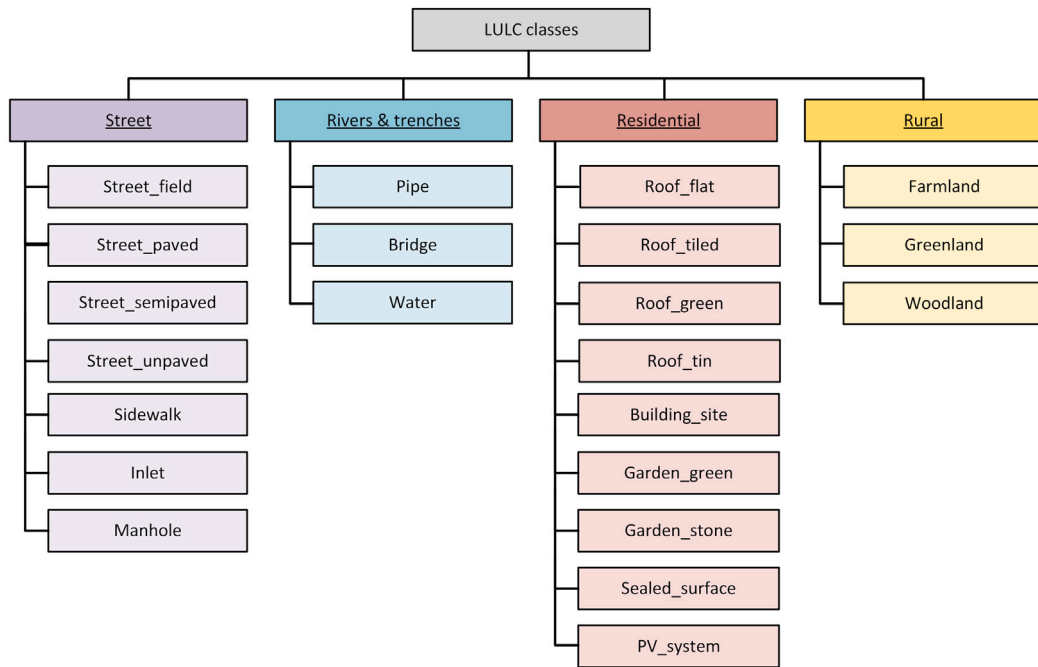


Fig. 2. Overview of the LULC classes grouped into four macro-classes.

assigned a specific object class. Deep neural network architectures perform best on popular benchmarks for this task, such as Cityscapes (Cordts et al., 2016) and ADE20K (Zhou et al., 2017). However, because of the typically large size and high resolution of an orthophoto, it cannot be directly fed into the neural network, which operates on smaller image sizes. Therefore, the orthophoto is first divided into patches of an appropriate size for the neural network, processed individually, and finally combined into a high-resolution LULC map.

This LULC map can then be used as basis to automate parts of the heavy rainfall hazard mapping process. In particular, segmented hydraulically significant structures, like inlets and manholes, can be utilized during mesh generation to construct a more accurate and up-to-date mesh of the environment. Moreover, segmented areas with different LULC classes can be employed during mesh parametrization to assign physical properties to corresponding mesh cells. However, fully automating the integration of the resulting LULC map into the mesh generation and parametrization processes requires significant further development. While this falls beyond the scope of the present study, it will be addressed in detail in future work.

2.1. Classification system

The first step towards an automatic LULC classification method is the definition of relevant object classes. For this purpose, we identified objects that are currently relevant for the mesh generation and mesh parametrization steps and which can be visually recognized in UAV images. Our current approach defines multiple macro-classes linked to different rule sets for mesh generation and mesh parametrization. For instance, streets are modeled differently than rivers and trenches during mesh generation, and streets exhibit different parameter sets than rural areas during mesh parametrization. Therefore, we grouped our classification system into four macro-classes and 22 LULC classes according to the manual process.¹ This classification system is depicted in Fig. 2, and example images of each class are detailed in Appendix A. A description of each macro-class and its LULC classes is given in the following:

1. **Street:** The first macro-class comprises classes that belong to street areas. These are relevant to correctly model the flow and infiltration of water during the mapping process. For example, different street materials affect the infiltration of water, and curbs (between a street and a sidewalk) can be extracted and influence the flow of water (Zamboni et al., 2024). Thus, these classes are relevant for both mesh generation and mesh parametrization. To this end, we distinguish between four different street materials (field, paved, semipaved, and unpaved) and sidewalks. Additionally, this macro-class comprises inlets and manholes since they are especially relevant for the flow of water into the sewerage system.

¹ The LULC classes are currently selected with an application focus on Central Europe.

Table 1
Camera parameters and flight concept.

Camera parameters	Aperture	F4
	Shutter speed	1/800–1/1000
	ISO	100–800
Flight concept	Flight speed	5.8 m/s
	Flight height	120 m, terrain following
	Side overlap	70%
	Forward overlap	80%
	Ground Sampling Distance	1.5 cm/px

Table 2
Summary of the 3D error on the 12 CPs.

Count	X error (cm)	Y error (cm)	Z error (cm)	XY error (cm)	Total (cm)
12	0.85	1.29	1.48	1.54	2.14

- Residential:** The second macro-class comprises objects and structures appearing on residential ground. Especially different types and materials of roofs have an impact on the water flow. Hence, this macro-class encompasses flat, tiled, green and tin roofs in addition to building sites and photovoltaic systems. Moreover, it is necessary to characterize the area around a building to accurately model the water infiltration during mesh parametrization. To this end, green and stone gardens are considered in addition to sealed surfaces.
- Rivers and trenches:** This macro-class consists of three classes dealing with rivers and trenches, namely pipes, bridges, and water. These object classes are important for the flow of water.
- Rural:** The last macro-class distinguishes between three rural classes, which are relevant for the infiltration of water. These classes are farmland, greenland, and woodland.

2.2. Data collection and preprocessing

Since there are no publicly available image datasets with these relevant LULC classes and a high-detail resolution, a novel dataset is created for this study.

2.2.1. UAV data acquisition

The data were collected using a DJI Matrice M300 RTK quadcopter UAV equipped with a P1-35 mm sensor to capture high-resolution, full-frame 45 MP RGB images with a pixel size of 4.4 μm . Real-time kinematic (RTK) positioning was employed for direct georeferencing of the images, supplemented by 9 Ground Control Points (GCPs) and 12 independent Check Points (CPs) to verify data quality. The survey was carried out on a small village in Central Europe, focusing on a newly developed residential area. The area was selected because of the presence of relevant object classes and its hydrological contribution, as it represents a newly developed urban landscape not yet integrated into current hydrological models. In total, an area of around 0.3 km^2 was covered. The data were captured in the late afternoon in autumn under suboptimal lighting conditions, resulting in long shadows and high ISO values. Most of the images were taken from nadir perspective (90°). At the end of the survey, oblique images were taken (45°) as well to ensure height optimization of the DEM. Information about camera parameters and flight concept can be found in [Table 1](#).

2.2.2. Data processing

Data processing was done with Agisoft Metashape Professional v2.0.1 in a standard Structure-from-Motion photogrammetry workflow ([Hendrickx et al., 2019](#); [Eltner et al., 2022](#)). The images were aligned with high accuracy and a key and tie point limit of 40,000 and 10,000, respectively, in the program settings. Depth maps were generated using high quality and mild filtering mode. A high-quality dense point cloud was generated from these depth maps, resulting in a point cloud of 400 million points. From this, a DEM and an orthophoto was created with a resolution of 3 cm and 1.5 cm, respectively, using the default blending mode “Mosaic”. Accuracy of the data was checked by evaluation of the 3D error on the 12 CPs, summarized in [Table 2](#). An overall accuracy of 2.14 cm was reached.

2.3. Annotated image dataset

The result of the previous step is an orthophoto with a size of 36.000 px $\times$ 42.000 px including most of the previously defined object classes. To utilize this orthophoto for automatic LULC classification and neural network training, it must be transformed into a high-quality, representative image dataset with annotations.

2.3.1. Dataset creation

Since we consider LULC classification as a semantic segmentation problem from the field of computer vision, each pixel of the orthophoto must be assigned to one of the specified LULC classes in Fig. 2. For this purpose, there are several annotation techniques, which can be grouped into three categories:

- **Manual:** This is the classical approach in which a human annotator labels images with arbitrary object classes. Due to the completely manual nature of this process, this approach is most time-consuming, but is not restricted to certain object classes.
- **Semi-automatic:** This approach integrates a computational model into the annotation process to support the human annotator, such as Segment Anything Model (SAM) (Kirillov et al., 2023). A model generates annotations for certain object classes, and a human annotator only needs to verify the automatic generations, which saves time. However, this approach is limited to object classes the model can recognize, and missing or wrong objects must be specified by a human annotator. Thus, the automatic annotation is limited to certain object classes, but can be mitigated by human intervention.
- **Automatic:** This approach provides full automatic image annotation with no human interaction in the annotation process. It combines several state-of-the-art foundation models, such as Recognize Anything Model (RAM) for image tagging and Grounded SAM for text-to-mask generation (Zhang et al., 2023; Ren et al., 2024). This approach is the most time-saving one, but is limited to general object classes recognizable by the models. Thus, it only works well for general-purpose problems.

Since we want to annotate specific (non-general) LULC classes and reduce the total annotation time, we pursue the semi-automatic annotation process, which is a trade-off between recognizing arbitrary objects and reducing annotation time. For this purpose, we employ the open source, web-based image annotation tool CVAT² in combination with SAM. This foundation model is an advanced model for image segmentation with “zero-shot” capabilities which makes it useful for supporting the image annotation process. However, the accuracy of SAM’s prediction does not always satisfy our high accuracy requirements, or sometimes a segmentation mask cannot be generated for certain structures. In these cases, an annotator must explicitly specify or adapt the segmentation mask.

This tooling is well suited to annotate images, but not directly high-resolution orthophotos due to memory limitations. Therefore, the orthophoto is first divided into 2000 px × 2000 px non-overlapping image patches, that fit into memory and that have an adequate size to be processed by SAM. Non-overlapping patches are chosen to avoid redundancy in the annotation process. Since the orthophoto also contains “empty” areas, especially at the margins, this process results into 264 images for the given orthophoto. These images are subsequently provided to the annotation tool. This annotation process consisted of multiple steps with the objective of creating high-quality annotations:

1. Provision of an annotation guideline including object classes and examples by hydraulic engineers
2. Semi-automated annotation using CVAT and SAM by a computer vision expert
3. Review of the annotations by hydraulic engineers
4. Incorporation of reviewers’ feedback by a computer vision expert

Afterwards, the annotated, non-overlapping image patches are merged into an annotated orthophoto.

This annotated orthophoto cannot be directly fed into a deep neural network for training because of its large size and memory limitations. Hence, the size of the annotated orthophoto must be reduced. To this end, we pursue a multi-scale image cropping approach in which image patches of different sizes are created to account for different object sizes, which is characteristic of our problem. The image sizes range between 256 px × 256 px and 2048 px × 2048 px. Moreover, to create a high-quality, representative image dataset, intra-class similarities should be limited. For this purpose, the number of single-annotation images per class is limited to a certain threshold. This is especially relevant for classes with a small intra-class variance in the dataset, such as “farmland” and “greenland”. In addition, the object classes “greenland” and “garden_green” are combined as they represent the same object class with the same hydrological properties but in different contexts, i.e. grass in rural and residential areas. In summary, this dataset creation process results in a total of more than 12K annotated images with different sizes as shown in Table 3. Exemplary images of the dataset including different sizes and representative objects are visualized in Fig. 3. It can be observed that images with larger

² <https://github.com/cvat-ai/cvat> [Online access: 08.03.2025].

Table 3

Number of images with different sizes of the multi-scale image dataset.

Image size	Number of images
2048 px $\times$ 2048 px	308
1024 px $\times$ 1024 px	1146
512 px $\times$ 512 px	2724
256 px $\times$ 256 px	8351
Total	12,529

**Fig. 3.** Example images of the dataset with decreasing sizes from left to right (scaled to the same size for visualization).

sizes encompass more contextual information, which is relevant for larger objects and structures, whereas images with smaller sizes keep the details of small objects.

2.3.2. Dataset characteristics

Since real-world scenarios often feature imbalanced data, it is important to understand the underlying structure of the data. For this purpose, the number of images and the frequency per object label is calculated as shown in Fig. 4. The plots show that the dataset is highly imbalanced featuring overrepresented classes, such as “greenland”, and underrepresented classes, such as “inlet”, “building site” and “water”. This is caused by the nature of the study area which is a residential area with surrounding rural areas, which leads to the majority of images with a greenland area. Moreover, small objects, such as inlets and manholes, have a relatively low frequency due to their small sizes. Additionally, there are four object classes that are not represented in the dataset, i.e. “bridge”, “pipe”, “green roof” and “green garden”. The object classes, which are missing in the dataset due to absence in the study area, are ignored in the remainder of this article.

2.4. Neural network architectures

State-of-the-art semantic segmentation algorithms are based on deep neural network architectures. Starting with the advent of deep learning architectures for semantic segmentation in 2015 (Long et al., 2015), these architectures became more sophisticated and improved their performance continuously (Wagner et al., 2023). Given that a key challenge of our problem is the segmentation of objects with varying sizes, we focus on state-of-the-art neural network architectures designed to address this issue in different ways. To this end, we selected eight promising architectures to identify the most suitable approach for our specific task. Several of these architectures have been successfully applied in UAV image segmentation (Osco et al., 2021) and are relatively simple compared to transformer-based models. Their lower complexity reduces the need for extensive training data, which is advantageous for this study. A summary of the selected architectures and their strategies for handling objects of varying sizes is provided in Table 4. These

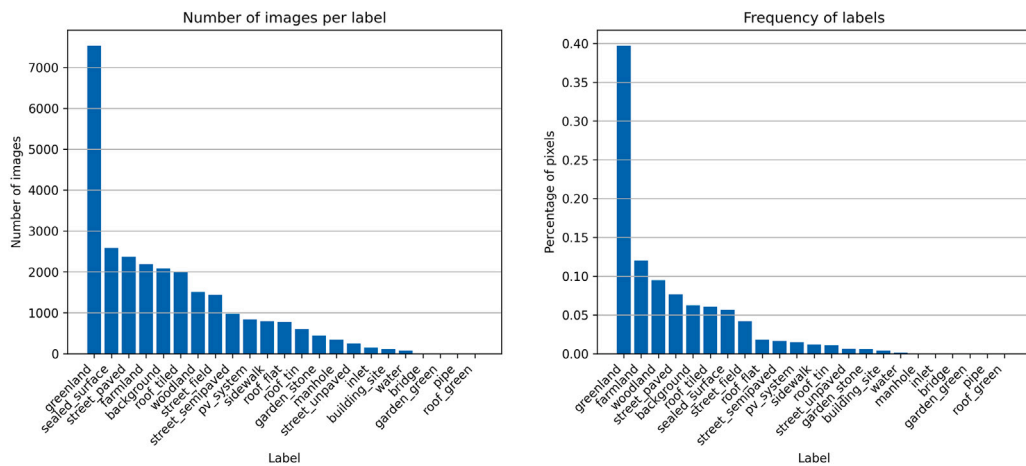


Fig. 4. Dataset characteristics including number of images (left) and frequency (right) depending on the object class.

Table 4

Overview of considered semantic segmentation architectures and their strategies to deal with objects of varying sizes.

Architecture	Context aggregation strategies
FCN	–
U-Net	–
PSPNet	Pyramid Pooling module
DeepLabv3+	Atrous Spatial Pyramid Pooling module
CFNet	Aggregated Co-occurrent Feature module
ACFNet	Attentional Class Feature module
DANet	Position Attention Module and Channel Attention module
OCNet	Object Context module

architectures and their main characteristics, in particular their context aggregation strategy, are briefly described in the following paragraphs, and the original publication is referenced for more details.

The first considered architecture is the FCN architecture (Long et al., 2015). The main idea of this work is to transform the fully connected layers of image classification networks into convolutional layers, allowing a classification network to output a dense heatmap instead of classification scores. To restore the spatial resolution of the segmentation maps, the authors introduce the concept of upsampling, which is performed using bilinear interpolation or learnable deconvolutional layers, also known as transposed convolutions. A disadvantage of this architecture is, that it does not implement a strategy to aggregate contextual information from multiple scales. However, since it was the first deep neural network designed for semantic segmentation, it serves as a baseline architecture in this study.

Another important architecture is the U-Net architecture (Ronneberger et al., 2015). The primary innovation of this work lies in its symmetric encoder–decoder structure, designed specifically for biomedical image segmentation. The encoder path follows a typical convolutional network pattern, using repeated application of convolution and max-pooling layers to capture context and reduce spatial dimensions. In contrast, the decoder path mirrors the encoder but uses transposed convolutions to upsample the feature maps, gradually restoring the original image resolution. A key characteristic of U-Net is the use of skip connections between the encoder and decoder layers. These connections concatenate feature maps from the encoder with the corresponding decoder layers, providing fine-grained details that help to improve segmentation accuracy. Its proven success in biomedical image segmentation makes it a compelling choice for various domains, including UAV imagery. Nonetheless, it also does not feature an integrated multi-scale context aggregation scheme.

While the first two architectures present baselines due to their popularity and success in other domains, the following architectures present distinct strategies to aggregate information from different scales. Thus, they are promising candidates for our problem dealing with various objects with a wide variety of sizes. For this purpose, the PSPNet architecture introduces the Pyramid Pooling module (Zhao et al., 2017). This module consists of parallel pooling layers with different bin sizes, allowing the network to capture both global and local context. It enables PSPNet to develop a comprehensive understanding of the global context within a scene. By pooling features from regions of varying sizes, the network can effectively handle objects with significant variations in scale and appearance, making it particularly interesting for recognizing diverse LULC classes with significant variations in scale in complex scenes.

The fourth architecture is DeepLabv3+ (Chen et al., 2018), an enhanced version of the DeepLabv3 architecture introduced by Chen et al. (2017). A key innovation in DeepLabv3+ is the combination of Atrous Spatial Pyramid Pooling and an encoder–decoder structure similar to U-Net. The Atrous Spatial Pyramid Pooling module employs parallel atrous convolutions with different

dilation rates, enabling the network to capture information at multiple scales. This multi-scale feature extraction is crucial for accurately segmenting objects of varying sizes and complexities. Hence, making it suitable for our use case.

Another neural network architecture is CFNet (Zhang et al., 2019c), which advances semantic segmentation by leveraging co-occurring features. This work introduces the Co-occurrent Feature Model, which predicts the distribution of co-occurring features for a given target. By utilizing the semantic context inherent in these features, the method incorporates an Aggregated Co-occurrent Feature module that aggregates these probabilities across the scene. It captures fine-grained, spatially invariant representations, effectively conveying co-occurrence context information across different spatial locations. In addition to this module, global features are extracted using global average pooling, followed by convolution and upsampling. Both branches are subsequently concatenated along the channel dimension and passed through a final convolution to create a segmentation mask. The incorporation of co-occurring features allows CFNet to resolve ambiguities in object classification, particularly in complex scenarios, which makes it particularly interesting for the given UAV imagery featuring highly complex scenes.

The ACFNet architecture introduces another concept to capture global context, called “class centers”, which capture global context from a categorical perspective (Zhang et al., 2019b.). The architecture employs an Attentional Class Feature module that adaptively combines these class centers based on the semantic context of each pixel, allowing the network to focus on relevant class features. These attentional class features are combined with the base network’s feature maps to predict a segmentation mask through a final convolution. This architecture is considered for evaluation because it features an alternative way to incorporate global context.

DANet integrates two attention mechanisms to capture local and global contextual information: position and channel attention modules (Fu et al., 2019). The position attention module enables the network to learn spatial interdependencies among features in the image, while the channel attention mechanism captures interdependencies across channel maps. This extension of the self-attention mechanism allows for the modeling of rich contextual relationships crucial for accurate segmentation. The authors embed these attention modules within a general framework, where a backbone image classification network extracts feature maps that are then fused to generate high-resolution segmentation masks. This presents another option to capture local and global contextual information.

Finally, OCNet offers a novel context aggregation approach focusing on the relationship between pixels within the same object category (Yuan et al., 2021). The architecture is built upon a standard convolutional neural network backbone, which extracts feature maps that are then processed through an object context module. The authors define the object context for each pixel in an image as the set of pixels belonging to the same object category and propose an efficient way for calculating the context. The output of the object context module is a contextual feature map, that is finally fed into a classifier to predict the final segmentation masks. The proposed context aggregation scheme is interesting for our use case as it offers an alternative approach to incorporate context information.

All of these semantic segmentation architectures are based on backbone neural networks for feature extraction, which can be typically replaced by other backbone architectures. In order to identify a best possible performing deep neural network for our problem, we evaluate four state-of-the-art backbone models for each semantic segmentation architecture. These backbone models were selected because they are established backbone architectures implemented in various software libraries and all provide pre-trained weights on ImageNet dataset (Deng et al., 2009), which is beneficial for ease of use and transfer learning. The backbone models are ResNet (He et al., 2016), Xception (Chollet, 2017), MobileNetV2 (Sandler et al., 2018) and EfficientNet (Tan and Le, 2019). Specifically, we use a ResNet101 for the ResNet architecture and an EfficientNetB4 for the EfficientNet architecture to account for computational constraints. Since not all of the semantic segmentation architectures are compatible with each backbone, this results in a total of 29 different neural network architectures. An overview of the architectures and configurations in the training process are summarized in Table 5.

2.5. Neural network training

This subsection details the training process of the deep neural networks on the novel image dataset.

2.5.1. Hardware and software

The basis for the semantic segmentation architectures is a Python library for high-level semantic segmentation models based on TensorFlow, including pretrained backbone models.³ This library was modified to meet our needs, such as updating the source code to be compatible with the latest TensorFlow versions. To facilitate the easy distribution of the experimental setup and ensure the reproducibility of the results, the implementation and training process are encapsulated in a Docker container. The software environment consisted of an Ubuntu operating system (v22.04), Docker Engine (v27.3.1), including the NVIDIA Container Toolkit for GPU support (v1.16.2), and TensorFlow library (v2.16.2). The neural network training was performed on a desktop PC that features an Intel Core i9-12900K processor, 32 GB of RAM, and an NVIDIA RTX A4000 graphics card with 16 GB of GPU memory.

³ <https://github.com/JanMarcelKezmann/TensorFlow-Advanced-Segmentation-Models> [Online access: 08.03.2025].

Table 5

Summary of the evaluated neural network architectures and configurations for the training process.

Architecture	Backbone	Parameters (M)	Input width/height (px)	Batch size
FCN	ResNet101	42.7	256	64
FCN	Xception	20.9	256	64
FCN	MobileNetV2	2.3	256	64
FCN	EfficientNetB4	17.7	256	64
U-Net	ResNet101	60.3	256	16
U-Net	MobileNetV2	15.4	256	16
U-Net	EfficientNetB4	34.0	256	16
PSPNet	ResNet101	43.0	264	16
PSPNet	MobileNetV2	2.7	264	16
PSPNet	EfficientNetB4	18.1	264	16
DeepLabv3+	ResNet101	44.9	256	16
DeepLabv3+	MobileNetV2	4.4	256	16
DeepLabv3+	EfficientNetB4	19.8	256	16
CFNet	ResNet101	48.9	256	16
CFNet	Xception	26.9	256	16
CFNet	MobileNetV2	6.5	256	16
CFNet	EfficientNetB4	23.2	256	16
ACFNet	ResNet101	45.4	256	16
ACFNet	Xception	23.8	256	16
ACFNet	MobileNetV2	4.6	256	16
ACFNet	EfficientNetB4	20.2	256	16
DANet	ResNet101	46.4	256	16
DANet	Xception	25.6	256	16
DANet	MobileNetV2	4.6	256	16
DANet	EfficientNetB4	20.6	256	16
OCNet	ResNet101	44.0	288	16
OCNet	Xception	22.8	288	16
OCNet	MobileNetV2	2.9	288	16
OCNet	EfficientNetB4	18.7	288	16

2.5.2. Data preparation

Before feeding the image dataset into the training process, the image data are prepared to ensure a valid training process and improve the generalizability of the trained neural networks. For this purpose, the dataset is divided into a training dataset (70%), a validation dataset (20%), and a test dataset (10%). The training dataset is used exclusively to optimize the neural network's weights, the validation dataset is used to calculate performance metrics during training, and the test dataset is used to evaluate the neural network after the training process. Thus, the test dataset is not seen by the neural network during training, allowing for a fair estimation of its performance and generalizability on unseen data.

In addition to the dataset split, data augmentation is performed on the training dataset to artificially increase the amount and variance of the data. To this end, [Xu et al. \(2023\)](#) distinguish between three main categories of image augmentation techniques:

1. **Model-free augmentation** does not rely on a pre-trained model and may utilize either a single image or multiple images. Single-image augmentation leverages traditional image processing including geometric transformations, color image processing, and intensity transformations. In contrast, multiple-image augmentation algorithms require multiple images and are further divided into instance- and non-instance-level techniques. Some methods of this category are rotations, flipping, blurring, adding noise, CutMix ([Yun et al., 2019](#)) and GridMix ([Baek et al., 2021](#)). The advantage of model-free approaches lies in their flexibility and independence from pre-trained models. They can be directly tailored to specific datasets and tasks, which can lead to better adaptation to the particular problem. Additionally, they are often less computationally intensive and allow for quicker implementation, as they do not require extensive preprocessing or model tuning.
2. **Model-based augmentation** necessitates the use of trained models to generate synthetic images. This augmentation technique can be classified as unconditional, label-conditional, or image-conditional. Unconditional image generation employs a generative model with noise as input, whereas label-conditional and image-conditional image generation require a specific label and images as additional inputs, respectively. The methods are typically based on variations of Generative Adversarial Networks (GAN) ([Goodfellow et al., 2014](#)). An advantage of this category of techniques is that they can produce new images and not just alter simple image features. However, this procedure is not as straightforward as the previous category because it requires training of an image generation model.
3. **Optimizing policy-based augmentation** focuses on the determination of optimal operations within a large parameter space. This category is further subdivided into reinforcement learning-based and adversarial learning-based methods. The reinforcement learning approach employs an agent to find an optimal policy in a search space, whereas adversarial learning-based methods are designed to intelligently create augmentations that challenge a trained model and thus contribute to its generalizability. Methods of this category are AutoAugment ([Cubuk et al., 2019](#)) and AdaTransform ([Tang et al., 2019](#)). An advantage of methods in this category is that they do not require domain knowledge since an augmentation policy is automatically determined. However, this procedure comes with complexity and significant computational costs.

In our study, the main challenges are geometric transformations due to the UAV flight and camera perspective as well as changes in lighting conditions due to different weather conditions and times of day. Therefore, we focus on geometric and color space augmentation techniques from the category of model-free augmentation techniques to address these challenges. These techniques are simple compared to methods of the other two augmentation categories, established in common software libraries and therefore easy to use for a robust implementation. Moreover, as these techniques are model-free, they do not require to train an image generation model or search a large parameter space first. This reduces the overall complexity of data augmentation and increases ease of use. Specifically, the following techniques are employed sequentially on the images:

1. **Geometric augmentation** is performed to account for possible geometric transformations, such as rotations and translations of the images. Specifically, we apply random flips (horizontal and vertical), random translations (10% of the image width and height in horizontal and vertical directions, respectively) and random rotations (up to 360°). Points outside the boundary of an input image are filled with “empty” pixels. These transformations are applied to the RGB images as well as the annotation masks. This approach is valid since the images were captured by a UAV, and these geometric augmentations generate images that could also be captured by the UAV in the real world.
2. **Color space augmentation** is used to account for possible variations in brightness and contrast in the images. Therefore, our color augmentation includes a random change in brightness (factor of 0.2) and a random modification of contrast (factor of 0.5). Both transformations are applied to the RGB images of the dataset. This approach is valid as these color transformations can also occur in the real world due to lighting changes and camera characteristics.

2.5.3. Training parameters

The hyperparameters for the training process were carefully selected to optimize the performance of the neural networks. The backbone layers of the models were not frozen, allowing for fine-tuning during the training process. The images were resized to a specified width and height to maintain consistency, e.g. 256 px × 256 px, and the batch size was determined based on the available computational resources and ranged between 16 and 64. Details of the image size and batch size for the various neural network architectures can be found in Table 5. The optimizer used was Adam (Kingma and Ba, 2015), configured with a learning rate of 0.001, and the loss function employed was cross-entropy between the annotation masks and predictions, which is suitable for multi-class classification tasks. Training was carried out over a maximum of 100 epochs, with a learning rate reduction by a factor of 0.2 if learning stagnated, and early stopping was incorporated to prevent overfitting. Additionally, training runs were performed using custom class weights to weight the loss function, encouraging the model to focus on certain object classes. Specifically, we gradually increased the weights up to a factor of 10 for object classes which featured an Intersection over Union (IoU) less than 0.6. This was particularly relevant for the underrepresented classes, as shown in Fig. 4.

2.5.4. Evaluation metrics

For the evaluation of the approaches to LULC classification, several metrics are employed to quantify the performance of the trained neural networks. Key metrics include accuracy, IoU, and Mean IoU, which are commonly used in the field of semantic segmentation.

To understand these metrics, the following terms are defined:

- *TP*: True Positives - the number of correctly classified positive pixels.
- *TN*: True Negatives - the number of correctly classified negative pixels.
- *FP*: False Positives - the number of incorrectly classified positive pixels.
- *FN*: False Negatives - the number of incorrectly classified negative pixels.

The accuracy computes the ratio between the number of correct classifications and total classifications and is calculated as follows:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Accuracy is simple to understand and compute, making it a widely used metric. However, it can be misleading in imbalanced datasets, as it may not reflect the true performance for minority classes.

To address this limitation, the IoU for specific target classes is defined as:

$$IoU = \frac{TP}{TP + FP + FN} \quad (2)$$

IoU provides a more nuanced measure of performance by considering both false positives and false negatives. Nonetheless, it can be sensitive to small changes in the predicted and actual areas, particularly in cases of small object detection.

Furthermore, the Mean IoU is calculated by determining the IoU for each class and taking the average:

$$Mean\ IoU = \frac{1}{n} \sum_{i=1}^n IoU_i \quad (3)$$

where n is the number of classes. Mean IoU offers a comprehensive assessment across all classes, emphasizing the performance on individual classes. On the downside, if some classes have very low IoU, they can significantly lower the Mean IoU, potentially overshadowing good performance in other classes.

Table 6

Overview of the performance of different models ordered by their accuracy in descending order. Columns containing IoU values are ordered by their mean value in descending order from left to right.

Architecture	Backbone	Accuracy	Mean	IoU								
				IoU	pv_system	roof_tiled	street_paved	background	...	garden_stone	manhole	water
FCN	EfficientNetB4	0.927	0.734	0.918	0.893	0.923	0.932	...	0.495	0.530	0.366	0.000
FCN	Xception	0.922	0.721	0.932	0.904	0.933	0.883	...	0.375	0.521	0.356	0.000
CFNet	EfficientNetB4	0.906	0.716	0.941	0.910	0.926	0.899	...	0.470	0.537	0.473	0.000
CFNet	Xception	0.906	0.695	0.919	0.892	0.926	0.890	...	0.351	0.527	0.340	0.000
FCN	ResNet101	0.904	0.656	0.898	0.879	0.906	0.882	...	0.310	0.217	0.170	0.000
FCN	MobileNetV2	0.891	0.639	0.915	0.887	0.895	0.874	...	0.396	0.197	0.019	0.000
DANet	EfficientNetB4	0.890	0.664	0.901	0.883	0.880	0.848	...	0.489	0.500	0.210	0.011
DANet	Xception	0.888	0.628	0.898	0.859	0.883	0.856	...	0.221	0.391	0.133	0.001
DANet	ResNet101	0.879	0.601	0.884	0.840	0.874	0.858	...	0.201	0.321	0.117	0.000
CFNet	MobileNetV2	0.874	0.648	0.908	0.843	0.842	0.851	...	0.480	0.374	0.544	0.000
OCNet	EfficientNetB4	0.869	0.587	0.849	0.826	0.849	0.816	...	0.237	0.294	0.096	0.000
CFNet	ResNet101	0.866	0.591	0.855	0.818	0.830	0.858	...	0.416	0.114	0.044	0.000
DeepLabv3	EfficientNetB4	0.857	0.591	0.800	0.850	0.831	0.901	...	0.335	0.464	0.101	0.000
DeepLabv3	MobileNetV2	0.849	0.537	0.818	0.791	0.851	0.843	...	0.284	0.049	0.000	0.000
DANet	MobileNetV2	0.842	0.530	0.799	0.823	0.795	0.768	...	0.147	0.164	0.021	0.000
U-Net	EfficientNetB4	0.838	0.479	0.705	0.781	0.811	0.809	...	0.074	0.000	0.000	0.000
OCNet	Xception	0.837	0.535	0.821	0.818	0.818	0.812	...	0.128	0.214	0.000	0.000
OCNet	ResNet101	0.826	0.498	0.785	0.788	0.816	0.836	...	0.121	0.041	0.023	0.000
U-Net	MobileNetV2	0.826	0.478	0.799	0.809	0.809	0.853	...	0.043	0.000	0.003	0.000
U-Net	ResNet101	0.820	0.476	0.759	0.798	0.806	0.869	...	0.066	0.000	0.000	0.000
ACFNet	EfficientNetB4	0.808	0.518	0.827	0.846	0.694	0.822	...	0.213	0.319	0.000	0.000
OCNet	MobileNetV2	0.798	0.447	0.782	0.769	0.776	0.777	...	0.050	0.013	0.000	0.000
DeepLabv3	ResNet101	0.794	0.483	0.801	0.806	0.842	0.840	...	0.150	0.030	0.017	0.000
ACFNet	ResNet101	0.777	0.468	0.862	0.833	0.832	0.746	...	0.195	0.058	0.000	0.000
PSPNet	EfficientNetB4	0.764	0.445	0.684	0.719	0.689	0.371	...	0.194	0.050	0.018	0.000
ACFNet	Xception	0.763	0.475	0.803	0.844	0.847	0.798	...	0.267	0.090	0.047	0.000
ACFNet	MobileNetV2	0.757	0.475	0.817	0.788	0.799	0.806	...	0.275	0.240	0.000	0.000
PSPNet	ResNet101	0.671	0.282	0.611	0.658	0.614	0.282	...	0.072	0.000	0.000	0.000
PSPNet	MobileNetV2	0.658	0.288	0.610	0.673	0.201	0.327	...	0.078	0.000	0.000	0.000

3. Results

This section presents the experimental results of the neural network training and comparison including custom class weights to target imbalanced data. Moreover, qualitative results and a comparison with similar works are presented.

3.1. Without custom class weights

The key evaluation results without a custom class weighting are summarized in [Table 6](#). This table highlights the performance of various neural network architectures in terms of accuracy and Mean IoU for selected LULC classes, specifically focusing on the four object classes that are best and worst recognized. The FCN architecture combined with EfficientNetB4 backbone stands out as the top performer, achieving the highest accuracy of 0.927 and a Mean IoU of 0.734. Most classes, including “pv_system”, “roof_tiled”, and “street_paved”, exhibit high IoU values, indicating effective classification. In contrast, the PSPNet architecture combined with MobileNetV2 backbone records the lowest accuracy at 0.658 and a Mean IoU of 0.288.

In terms of architecture comparison, FCN models, particularly those utilizing EfficientNetB4, consistently outperform other architectures such as DANet and U-Net. CFNet also shows commendable results, especially when paired with EfficientNetB4. This trend is further confirmed when investigating the best backbone model for each semantic segmentation architecture, where EfficientNetB4 consistently outperforms other backbone models.

Notably, classes such as “water” and “inlet” demonstrate low IoU values across numerous instances, highlighting significant challenges in their classification. The consistently poor performance in identifying “inlet” is particularly striking, as it was recognized by almost none of the tested architectures. The “water” class, while detected more frequently than “inlet”, still exhibits lower IoU values compared to other classes. This indicates potential difficulties in accurately delineating water bodies in high-resolution UAV imagery.

Overall, the results show that deeper backbone models, such as EfficientNetB4, typically produce better classification results. The results for all object classes can be found in the [Appendix](#) in [Table B.13](#).

3.2. With custom class weights

The performance of different models trained with custom class weights is summarized in [Table 7](#), where EfficientNetB4 is chosen as the backbone for all models, as it performed best across all architectures in the previous training attempt. This table highlights the accuracy and Mean IoU for various object classes. Notably, the implementation of custom class weights has led to improvements of IoU values in many cases, particularly for classes that were previously poorly recognized.

Table 7

Performance of different models trained with custom class weights ordered by their accuracy in descending order. EfficientNetB4 is chosen as backbone for all models. IoU values are shown for the object classes, and values in brackets indicate change compared to the training without custom class weights.

Architecture	Accuracy	Mean IoU	background	building_site	farmland	garden_stone	greenland
FCN	0.927 (± 0.000)	0.756 (+0.022)	0.944 (+0.012)	0.900 (-0.014)	0.873 (+0.001)	0.615 (+0.120)	0.823 (+0.002)
CFNet	0.911 (+0.005)	0.738 (+0.022)	0.883 (-0.016)	0.755 (-0.130)	0.854 (-0.005)	0.574 (+0.104)	0.797 (+0.032)
DANet	0.880 (-0.010)	0.668 (+0.004)	0.897 (+0.049)	0.620 (-0.189)	0.802 (-0.031)	0.401 (-0.088)	0.737 (-0.012)
DeepLabv3+	0.861 (+0.004)	0.599 (+0.008)	0.896 (-0.005)	0.250 (-0.406)	0.805 (± 0.000)	0.330 (-0.005)	0.679 (+0.006)
OCNet	0.860 (-0.009)	0.602 (+0.015)	0.835 (+0.019)	0.402 (-0.310)	0.813 (-0.007)	0.327 (+0.090)	0.660 (-0.017)
U-Net	0.818 (-0.020)	0.504 (+0.025)	0.849 (+0.040)	0.125 (-0.045)	0.766 (-0.038)	0.232 (+0.158)	0.650 (-0.036)
ACFNet	0.773 (-0.035)	0.545 (+0.027)	0.798 (-0.024)	0.322 (+0.083)	0.666 (-0.124)	0.347 (+0.134)	0.514 (-0.067)
PSPNet	0.771 (+0.007)	0.441 (-0.004)	0.470 (+0.099)	0.306 (-0.158)	0.706 (+0.088)	0.149 (-0.045)	0.575 (+0.022)
Architecture	inlet	manhole	pv_system	roof_flat	roof_tiled	roof_tin	sealed_surface
FCN	0.308 (+0.308)	0.516 (-0.014)	0.916 (-0.002)	0.762 (-0.031)	0.901 (+0.008)	0.643 (-0.041)	0.775 (+0.009)
CFNet	0.536 (+0.536)	0.676 (+0.139)	0.921 (-0.020)	0.712 (-0.047)	0.885 (-0.025)	0.572 (-0.077)	0.724 (-0.040)
DANet	0.509 (+0.498)	0.609 (+0.109)	0.895 (-0.006)	0.647 (-0.034)	0.876 (-0.007)	0.612 (-0.053)	0.655 (-0.053)
DeepLabv3+	0.367 (+0.367)	0.405 (-0.059)	0.858 (+0.058)	0.664 (+0.060)	0.838 (-0.012)	0.475 (-0.081)	0.630 (-0.014)
OCNet	0.481 (+0.481)	0.386 (+0.092)	0.853 (+0.004)	0.543 (-0.040)	0.820 (-0.006)	0.523 (-0.007)	0.631 (+0.010)
U-Net	0.000 (± 0.000)	0.431 (+0.431)	0.691 (-0.014)	0.493 (-0.006)	0.798 (+0.017)	0.423 (+0.014)	0.500 (-0.049)
ACFNet	0.428 (+0.428)	0.370 (+0.051)	0.848 (+0.021)	0.612 (+0.051)	0.851 (+0.005)	0.533 (+0.013)	0.645 (+0.019)
PSPNet	0.000 (± 0.000)	0.125 (+0.075)	0.651 (-0.033)	0.514 (-0.032)	0.714 (-0.005)	0.411 (-0.049)	0.418 (-0.022)
Architecture	sidewalk	street_field	street_paved	street_sempaved	street_unpaved	water	woodland
FCN	0.727 (-0.035)	0.763 (± 0.000)	0.937 (+0.014)	0.759 (-0.006)	0.890 (+0.004)	0.469 (+0.103)	0.839 (-0.017)
CFNet	0.625 (-0.050)	0.686 (-0.013)	0.923 (-0.003)	0.757 (-0.009)	0.869 (+0.005)	0.450 (-0.023)	0.829 (+0.064)
DANet	0.548 (-0.008)	0.676 (-0.017)	0.885 (+0.005)	0.644 (-0.047)	0.776 (+0.014)	0.152 (-0.058)	0.750 (+0.010)
DeepLabv3+	0.470 (+0.065)	0.599 (+0.115)	0.778 (-0.053)	0.622 (-0.007)	0.784 (-0.014)	0.184 (+0.083)	0.740 (+0.054)
OCNet	0.380 (-0.053)	0.645 (-0.041)	0.828 (-0.021)	0.659 (+0.046)	0.727 (-0.046)	0.180 (+0.084)	0.746 (+0.014)
U-Net	0.263 (+0.010)	0.627 (-0.050)	0.758 (-0.053)	0.550 (-0.040)	0.588 (-0.088)	0.113 (+0.113)	0.714 (+0.006)
ACFNet	0.504 (+0.013)	0.321 (-0.214)	0.832 (+0.138)	0.637 (+0.063)	0.850 (+0.013)	0.200 (+0.200)	0.079 (+0.079)
PSPNet	0.266 (-0.044)	0.547 (-0.018)	0.728 (+0.039)	0.440 (-0.022)	0.711 (-0.052)	0.068 (+0.050)	0.571 (-0.010)

The FCN architecture continues to stand out as the top performer, achieving an accuracy of 0.927 and a Mean IoU of 0.756. It detects the “manhole” class with an IoU of 0.516, a minor decrease compared to the previous model, and the “inlet” class with an IoU of 0.308, reflecting a significant increase. Following closely is CFNet, which achieves an accuracy of 0.911 and a Mean IoU of 0.738. CFNet detects “manholes” with an IoU of 0.676, showing an improvement of 0.139, and “inlets” with an IoU of 0.536 demonstrating commendable results.

In general, these findings underscore the value of custom class weights in improving classification accuracy for underrepresented classes. In particular, CFNet outperforms FCN in detecting smaller objects, such as “manholes” and “inlets”, which are particularly relevant for mesh generation in heavy rainfall hazard mapping. Additionally, CFNet achieves an overall accuracy and Mean IoU on the same level as FCN, making CFNet with the EfficientNetB4 backbone a promising candidate for further development in LULC classification applications.

3.3. Qualitative performance

Qualitative results of all semantic segmentation architectures with their best backbone model, i.e. EfficientNetB4 for all architectures, are visualized in the [Appendix](#) in [Figs. B.8](#) and [B.9](#). These reveal that CFNet and FCN usually achieve good segmentation results, while other architectures struggle with the classification of small objects and precise segmentation masks. In a direct comparison between CFNet and FCN, CFNet provides more accurate object boundaries.

More detailed qualitative results of the CFNet architecture with EfficientNetB4 backbone are visualized in [Fig. 5](#). The results demonstrate that the neural network performs well on larger structures and objects, such as buildings and streets, as well as on smaller objects, such as inlets and manholes.

However, the trained neural network also reveals limitations, as illustrated in [Fig. 6](#). For instance, the model sometimes fails to distinguish between object classes that are visually similar or even identical but have different land uses, as shown in the first rows of the figure. In these cases, the model cannot differentiate between trees in a green garden and those in a woodland, or between a sidewalk and a sealed surface on residential grounds. Moreover, occlusions negatively impact the segmentation results, as demonstrated in the second row of the figure, in which a camper and trees occlude the actual object classes, i.e. a semi-paved street and a sealed surface. Finally, the last row of the figure presents two examples in which the model completely fails to correctly segment the scene, with accuracies of 0.138 and 0.113.

In addition to the segmentation of the image patches, the final step of the proposed UAV-based LULC classification system, as illustrated in [Fig. 1](#), is the merging of the segmented image patches into a segmented orthophoto. This was done for the orthophoto of the study area, and the result is depicted in [Fig. 7](#) with different zoom levels for visualization purposes. The first row shows the lowest zoom level, revealing that the final model, consisting of a CFNet with an EfficientNetB4 backbone, can correctly classify large structures, particularly related to rural classes such as “farmland”, “greenland”, and “woodland”. When zooming into the orthophoto, as shown in the second row for the yellow rectangular area, it can be seen that medium-sized objects and structures

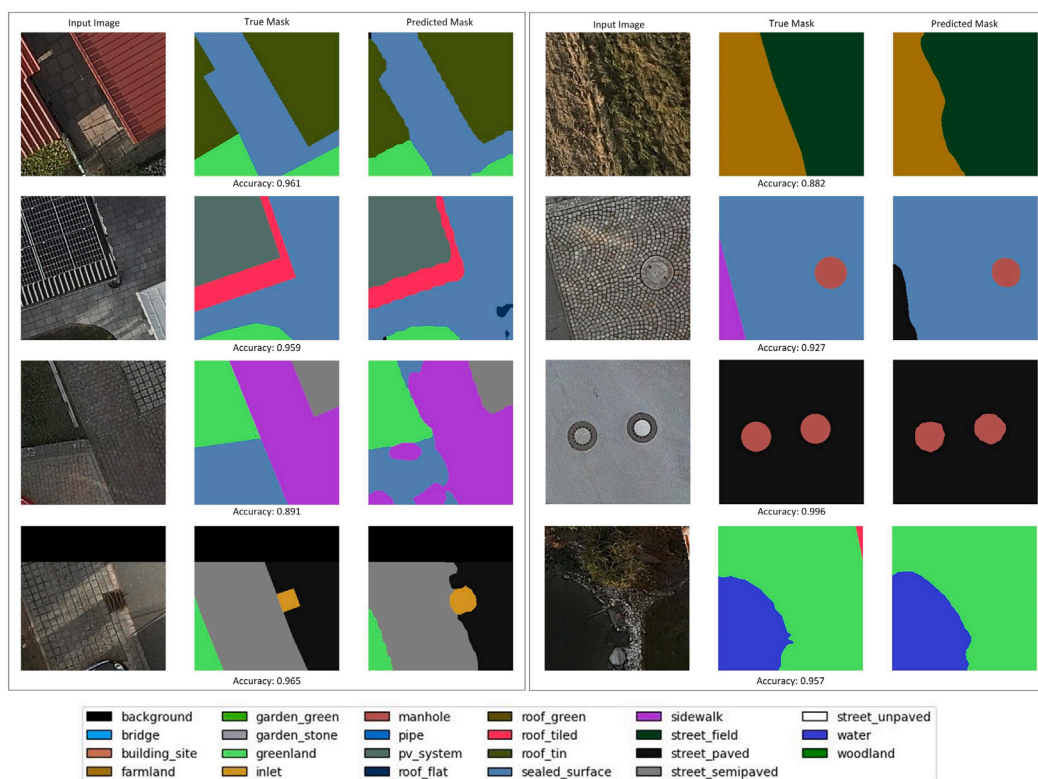


Fig. 5. Examples showing good segmentation results of the CFNet architecture with EfficientNetB4 backbone on images of the test dataset. Columns indicate the input image (left), ground truth mask (middle) and predicted mask (right), while rows show two samples per row.

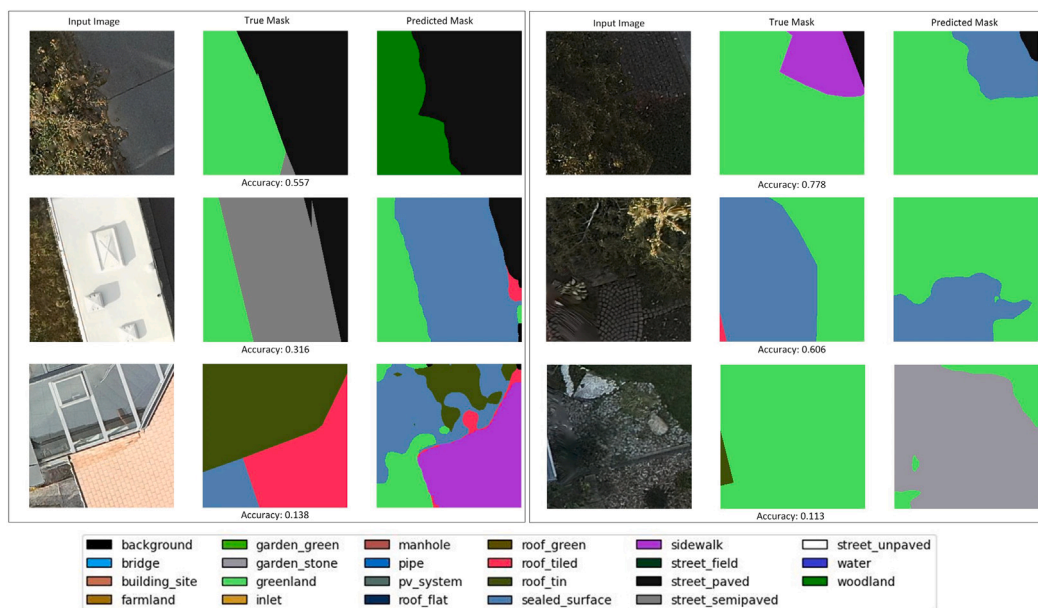


Fig. 6. Examples showing limitations of the CFNet architecture with EfficientNetB4 backbone on images of the test dataset. Columns indicate the input image (left), ground truth mask (middle) and predicted mask (right), while rows show two samples per row.

such as building roofs, ponds, and surfaces in residential areas are segmented with high accuracy. Finally, the last zoom level, shown in the bottom row for the blue rectangular area, highlights the correct classification of small objects, particularly inlets and manholes.

Table 8

Performance of our CFNet with EfficientNetB4 backbone trained with custom class weights compared to related works dealing with semantic segmentation on UAV images.

Work	Model	Dataset	Number of classes	Accuracy	Mean IoU
Girisha et al. (2019)	U-Net	ManipalUAVid	4	0.930	0.860
Giang et al. (2020)	U-Net	Custom	6	0.848	–
Rahnemoonfar et al. (2021)	PSPNet	FloodNet	9	–	0.804
Wang et al. (2022)	UNetFormer	UAVid	8	–	0.678
Wagner and Egerer (2022)	Random forest	Custom	6	0.796	–
Yi et al. (2023)	UAVFormer	UAVid	8	–	0.532
Cai et al. (2023)	UperNet	VDD	7	0.942	0.847
Our	CFNet	Custom	22	0.911	0.738

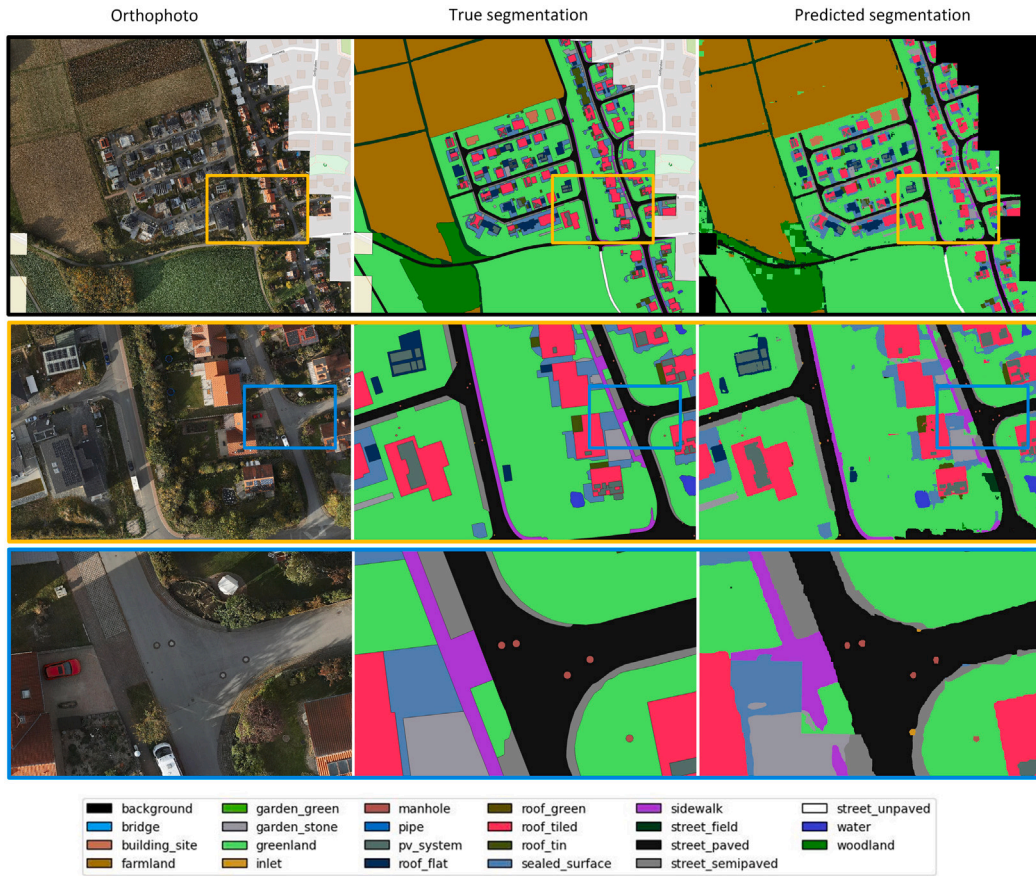


Fig. 7. Exemplary results of the CFNet architecture with EfficientNetB4 backbone on the overall orthophoto. Columns indicate the input orthophoto (left), ground truth segmentation (middle) and predicted segmentation (right). Yellow and blue rectangles illustrate regions of the orthophoto in the following rows.

Interestingly, two inlets have been overseen during the manual annotation process due to their small size, partial occlusions and challenging lighting conditions, but were found by the trained model.

3.4. Comparison with related works

Additionally, we compare our results with related works dealing with semantic segmentation of UAV image data, and present the results in Table 8. The results indicate a wide performance range, with accuracy varying from 0.796 to 0.942 and Mean IoU from 0.532 to 0.860. Our approach, achieving an accuracy of 0.911 and a Mean IoU of 0.738, falls within the mid-range. However, it addresses significantly greater complexity, handling 22 object classes compared to 4 to 8 in other studies.

4. Discussion

In this section, we discuss our findings in detail, examining the implications of our results and the various factors that influenced the performance of our LULC classification approach. We begin by analyzing the impact of different backbone models on the

overall performance of the neural networks, highlighting how specific architectures contributed to the accuracy and robustness of our classification method. Following this, we explore the effect of custom class weights, which were introduced to address data imbalances, and assess their effectiveness in improving the recognition of underrepresented classes. The impact of different semantic segmentation architectures on performance is discussed, and we compare our method to related works. We also address the limitations of our method, identify areas where further improvements are needed and suggest possible solutions. Finally, we discuss the relevance of our results for automating the process of heavy rainfall hazard mapping.

4.1. Impact of different backbone models

The choice of backbone model in neural network architectures significantly impacts the performance of LULC classification. As demonstrated in Table 6, different backbone models exhibit varying degrees of effectiveness in terms of classification accuracy and Mean IoU.

EfficientNetB4 consistently emerges as the top-performing backbone across all semantic segmentation architectures. For instance, the FCN architecture with EfficientNetB4 achieved the highest accuracy of 0.927 and a Mean IoU of 0.734 without custom class weights. The superior performance of EfficientNetB4 can be attributed to its compound scaling method, which balances network depth, width, and resolution, leading to better feature extraction and representation. In particular, Tan and Le (2019) observed, through visualizing class activation maps, that models with compound scaling are able to focus on more relevant regions with greater object detail compared to models scaled in only one dimension (depth, width, or resolution). Hence, this technique is especially suitable for complex scenes requiring a detailed feature representation. This characteristic makes it useful for our LULC classification problem to deal with the complex scenes and handle the diverse range of LULC classes effectively.

In contrast, backbones such as MobileNetV2 and ResNet101 exhibited lower overall performance. The PSPNet architecture with MobileNetV2, for example, recorded the lowest accuracy of 0.658 and a Mean IoU of 0.288 without custom class weights. This suggests that lighter and older backbone models may not be as effective in capturing the intricate details necessary for high-resolution UAV imagery segmentation. ResNet, in particular, is an architecture that scales up by increasing depth while keeping width and image resolution unchanged. This limitation makes it challenging for the model to capture all the objects and details in an image (Tan and Le, 2019). As a result, the missing detailed features cannot be utilized by the semantic segmentation architecture to effectively segment all objects, particularly small ones and intricate structures. This makes ResNet less suitable for complex LULC classification tasks.

This impact of backbone models is further evident when considering the performance on specific object classes. EfficientNetB4 demonstrated superior IoU values for object classes such as “pv_system”, “roof_tiled”, and “street_paved”, indicating its robustness in classifying both common and complex features. Conversely, models with MobileNetV2 and ResNet101 backbones struggled with lower IoU values, particularly for smaller objects.

Overall, the findings highlight the critical role of backbone models in determining the success of LULC classification tasks using deep neural networks. EfficientNetB4 stands out as a highly effective backbone, offering superior performance across a range of metrics and object classes. Future research could explore further optimization of backbone models and investigate the integration of newer architectures to enhance classification accuracy and robustness in high-resolution UAV imagery applications.

4.2. Impact of custom class weights

The implementation of custom class weights in the training of neural networks has shown a significant impact on the overall performance, particularly for underrepresented classes. The results, as summarized in Table 7, indicate that custom class weights can enhance the IoU for many object classes.

One of the most notable improvements is observed in the “inlet” class, which previously exhibited very low IoU values across all architectures. With the application of custom class weights, the IoU for the “inlet” class increased substantially, reaching 0.536 for the CFNet architecture. This demonstrates the effectiveness of weight adjustments in addressing class imbalance and improving the recognition of underrepresented categories.

The FCN architecture, when combined with EfficientNetB4 and custom class weights, still achieved the best performance with an increase of the Mean IoU of 0.002. Similarly, CFNet with EfficientNetB4 also showed marked improvements, particularly in detecting smaller and more challenging objects such as “manhole” and “inlet”.

However, it is important to note that the benefits of custom class weights are not universal across all architectures and classes. For example, some models, like DANet and OCNet, experienced a decrease in accuracy or IoU for certain classes. This indicates that the application of custom weights may need to be carefully tailored to the specific characteristics of each model and the target classes to achieve optimal results.

The challenges associated with the classification of the “water” and “inlet” classes remain, even with the application of custom class weights. Although there were improvements, the limitations of the training data for these classes may have constrained the extent of the enhancements. However, it is noted that these are small objects, which means that the IoU is sensitive to small changes in the predicted and actual classes. Thus, leading to small IoU values even if parts of the small object are correctly classified.

Overall, the findings underscore the value of custom class weights in improving classification performance for underrepresented classes in high-resolution UAV imagery. The substantial improvements observed for certain classes highlight the potential of this approach in addressing class imbalance and enhancing model robustness. Future work could focus on further refining the weighting strategy and exploring additional techniques to address persistent challenges associated with specific object classes, such as artificially increasing the frequency of underrepresented object classes in the dataset using Generative Artificial Intelligence (GenAI) methods.

4.3. Impact of different semantic segmentation architectures

When comparing the results of the different semantic segmentation architectures with EfficientNetB4 backbone and custom class weights, significant differences between the architectures are evident, as shown in [Table 7](#).

These can be roughly clustered into three groups based on their segmentation performance. The best performing group comprises FCN and CFNet, which achieve accuracies of 0.927 and 0.911. The superior performance of FCN is particularly surprising, as it does not implement a strategy for aggregating contextual information from multiple scales, which would typically be beneficial for complex LULC classification. A reason could be that FCN is the architecture with the fewest parameters, which could benefit from limited training data. However, it is notable that its performance is inferior to CFNet for small object classes, such as “inlet” and “manhole”, which could benefit from a multi-scale aggregation scheme. Since these classes are underrepresented in the dataset, their impact on overall accuracy is minimal, which explains the strong performance of FCN despite the lack of a multi-scale aggregation strategy. CFNet is on a similar performance level as FCN, but it features better IoU values for small object classes. This can be attributed to its Aggregated Co-occurent Feature module, which captures co-occurrence context information across different spatial locations. Thus, this approach does not only integrate a rough global context, but also a co-occurrence property among objects to resolve ambiguities in challenging scenes. This is particularly interesting for LULC classification as the presence of one object can be an indicator for another object. For instance, manholes and inlets are typically surrounded by streets and PV systems are often related to roofs. Hence, this enables the effective segmentation of objects in complex scenes. Compared to the other approaches integrating a multi-scale aggregation scheme, this strategy reaches the best performance, which makes it suitable for complex LULC classification.

The second best group encompasses DANet, DeepLabv3+ and OCNet with accuracies ranging between 0.860 and 0.880. These are all architectures implementing specific context aggregation strategies to deal with objects of varying sizes. DANet integrates two attention mechanisms, DeepLabv3+ combines Atrous Spatial Pyramid Pooling with an encoder–decoder structure and OCNet employs an object context module. Their results are strong, but they do not reach the performance of the first group. This can be attributed to the specific context aggregation strategies, which are not as effective as the Aggregated Co-occurent Feature module of CFNet. For instance, the Atrous Spatial Pyramid Pooling module integrates global context at different scales, which is beneficial to recognize objects of different sizes, but it does not facilitate the resolution of ambiguities in complex scenes.

The worst performing group includes U-Net, ACFNet and PSPNet. U-Net is the oldest architecture in this group and does not integrate a multi-scale aggregation strategy. Compared to FCN, which is also a simple architecture without a dedicated context aggregation strategy, U-Net performs significantly worse, even for larger structures. A reason could be that U-Net has nearly twice as much parameters as FCN, which requires more training data. PSPNet performs the worst of all architectures with an accuracy of 0.771. Considering the architectures with a context aggregation strategy, PSPNet is the oldest one, which implements a Pyramid Pooling module with four different scales and a final upsampling. There is no encoder–decoder structure to accurately restore the original resolution and enable a high-detail segmentation mask. Thus, this is a relatively simple approach to deal with contextual information, which could be a reason for the weak performance.

The observations in the quantitative results are also evident in the qualitative comparison of the segmentation results in the [Appendix](#) in [Figs. B.8](#) and [B.9](#). For instance, FCN successfully segments large structures, but fails to delineate smooth edges and fine details. In contrast to this, CFNet can more precisely segment the objects and performs best on small objects as shown in the last three rows of [Fig. B.9](#). The results of the architectures in the third group output significantly worse segmentation masks, including missing details and wrong classifications.

4.4. Comparison with related works

The results presented in [Section 3.4](#) show that our approach performs in the mid-range while handling significantly more object classes than other methods. Since fair comparisons require identical datasets, images, and object classes, direct comparison is not possible. However, the comparison provides insight into how well an approach performs on a given dataset. Notably, no other approach addresses such a wide variety of object classes, indicating a more complex problem. Despite this, our approach achieves relatively strong performance with an accuracy of 0.911 and a Mean IoU of 0.738.

4.5. Limitations

In spite of the overall effectiveness of the proposed approach for LULC classification, several limitations have been identified, as shown in [Fig. 6](#). One of the primary limitations is the model’s difficulty in distinguishing between visually similar object classes with different land uses. For example, the neural network sometimes fails to differentiate between trees in a green garden and those in a woodland, or between a sidewalk and a sealed surface on residential grounds. This indicates that the model may struggle with context-based distinctions, which are critical for accurate land use classification. The process of slicing the orthophoto into image patches, necessary for the computability of the neural network, reduces contextual information and exacerbates this issue.

Occlusions present another significant challenge. When objects such as campers or trees occlude the actual target classes, segmentation accuracy drops considerably. For instance, in the second row of [Fig. 6](#), occlusions lead to incorrect classifications of a semi-paved street and a sealed surface. This limitation highlights the need for more sophisticated approaches to handle occlusions, possibly through the integration of additional contextual information as described earlier. Alternatively, flying with UAV during the winter months when trees have no leaves could reduce the negative impact of tree occlusions.

Furthermore, the model exhibits notable failures in some cases, as shown in the last row of Fig. 6, where the segmentation accuracies are 0.138 and 0.113. These failures suggest that there are still scenarios where the model's performance is insufficient. In the left example in the bottom row, the model particularly fails to classify roofs correctly due to visual similarities with other object classes and a lack of wider context. Future work could address this issue by incorporating elevation information provided by the UAV. This could help to easily distinguish between visually similar classes that have different heights, such as roofs and objects on the ground. Additionally, inaccuracies in the annotation of the training data contribute to false classifications, as shown in the right example in the bottom row in which the model predicts a stone garden, but the annotation mask indicates a green garden. In this case, the model's prediction is correct when examining the input image. These inaccuracies in the image dataset should be fixed in future iterations by applying the model in an iterative refinement step of the annotations. Subsequently, the neural networks can be retrained on this improved image dataset.

Another limitation arises in the merging of segmented image patches into a complete orthophoto in the final step of the processing pipeline. While the model performs well on large and medium-sized structures, as shown in Fig. 7, there can be minor inconsistencies at the boundaries of patches. These inconsistencies may lead to artifacts in the final segmented orthophoto, which affects the overall quality. Future work could address this issue with more sophisticated merging techniques, such as interpolation between patches or the use of overlapping patches.

Moreover, while custom class weights improve the recognition of underrepresented classes, they do not completely resolve the classification challenges for all object classes. For example, the "water" class, despite improvements, still presents difficulties in accurate delineation. This suggests that further refinement of class weights or alternative strategies, such as the use of GenAI methods to augment the training data, may be necessary.

Finally, techniques need to be implemented to deal with changes in the LULC distribution, which are common when transferring to different geographical regions and time periods. To this end, active learning strategies play an important role, which deal with maximizing the performance gain of a model while annotating the fewest samples (Ren et al., 2021). In specific, a human-in-the-loop system could incorporate a human's feedback on selected unlabeled data to adapt the LULC classification model to geographic and temporal changes in LULC distribution.

4.6. Relevance for advancing heavy rainfall hazard mapping

The presented results show that it is possible to create high-resolution LULC maps with an accuracy exceeding 91%, particularly using a CFNet with an EfficientNetB4 backbone model. This is already a benefit for heavy rainfall hazard mapping since such detailed and up-to-date LULC information are currently not available. These information can be utilized in a next step to facilitate mesh generation and mesh parametrization of the hazard mapping process. For example, detected manholes can be integrated into the mesh of the environment, and physical properties of segmented LULC areas can be assigned to corresponding mesh cells. Even if the process of integrating the LULC maps into the hazard mapping process were to be carried out manually in a first attempt, this would make the labor-intensive field verification obsolete, lead to more accurate hazard maps and thus to better risk management. However, the ultimate goal is to fully automate this integration process without manual intervention. Thus, the mesh structure would be automatically modified according to detected relevant object classes, and mesh cells would be automatically assigned to the corresponding physical values of LULC classes. This would additionally lead to a significant reduction of creation time of hazard maps.

To this end, it is necessary to develop post-processing algorithms to improve the segmentation result to deal with remaining inaccuracies in the LULC maps, e.g. filling small gaps inside geometries or merging interrupted areas of the same geometry. This can be accomplished by morphological operations and integrating prior knowledge about object sizes and shapes from the field of image processing (Gonzalez and Woods, 2017).

Moreover, it is necessary to develop methods to automatically convert the LULC maps into required data structures for mesh generation according to different rule sets for the macro-classes and to seamlessly integrate them into the existing mesh. For instance, line segments of streets are differently modeled than rivers and trenches, and these must be integrated into the existing mesh without creating undesirable artifacts, such as gaps and peaks in the mesh. To this end, significant additional processing steps are necessary, such as buffering the line segments of streets, subdividing them based on longitudinal mesh element lengths, and defining appropriate mesh widths. Moreover, the delete tolerance of the surrounding triangular mesh elements must be adjusted to increase mesh density in areas like streets, rivers, or trenches according to model-specific requirements. Parameters such as mesh element length, width, and delete tolerance must be dynamically adapted depending on the type of structure to ensure accurate and stable mesh integration.

In addition, the integration of the LULC maps into the mesh parametrization process requires the assignment of specific hydrological and surface properties to different LULC types. Regarding parametrization, LULC maps are needed to assign Strickler roughness coefficients and CN values when defining surface roughness and runoff coefficients, respectively. The determination of the runoff coefficients is carried out using the Soil Conservation Service Curve Number (SCS-CN) method, which calculates the effective rainfall contributing to runoff based on soil types, land uses, and hydrological conditions. It was introduced in 1954 by the US Department of Agriculture SCS (Rallison, 1980) and was defined in the SCS National Engineering Handbook (NEH-4) (Ponce and Hawkins, 1996). The CN value is a key parameter of this method and is influenced by soil characteristics, land use, infiltration capacity, and antecedent moisture condition. CN values are determined using hydrological soil groups and LULC types such as agricultural areas, forests and urban zones (Zeng et al., 2017).

While the presented LULC classification results provide a solid foundation for automating hazard mapping, further substantial research is needed to address the complex task of integrating the LULC maps into the mesh generation and mesh parametrization steps. Therefore, details of this integration process will be addressed in future work.

5. Conclusions

In this study, we presented a UAV-based LULC classification method for the creation of high-resolution LULC maps to facilitate the construction of heavy rainfall hazard maps. Our method leverages state-of-the-art deep neural network architectures for semantic segmentation, addressing the challenges posed by imbalanced data and the diverse range of objects with highly varying sizes. The results demonstrate that the CFNet architecture with EfficientNetB4 backbone offers the best performance for this use case, achieving high accuracy and Mean IoU across various LULC classes. The implementation of custom class weights significantly improved the recognition of underrepresented classes, highlighting the importance of addressing class imbalances in training data. Qualitative analysis showed that our approach performs well in segmenting large structures, medium-sized objects, and even small objects such as inlets and manholes.

This establishes a solid foundation for automating the process of heavy rainfall hazard mapping. In particular, the resulting high-resolution and up-to-date LULC maps can be used to integrate relevant LULC structures into a mesh of the environment, enabling more accurate flood simulations. Moreover, physical properties of LULC structures, such as CN values, can be assigned to mesh cells without the need for manual intervention. Thus, this would be a significant step towards improving the speed of creation and accuracy of heavy rainfall hazard maps, which ultimately contributes to better risk management in the face of climate change.

However, certain limitations remain for the LULC classification method, particularly in distinguishing visually similar classes and handling occlusions. Future work should address the issue of underrepresented or missing classes in the dataset, such as pipes and bridges. The dataset could be expanded through additional UAV surveys or by employing GenAI methods to artificially increase the frequency of underrepresented classes. In addition, the evaluation of the proposed method on diverse datasets from different geographical regions and under various environmental conditions is crucial to better understand the generalizability and robustness of the model. Finally, further research should focus on how the results of the proposed LULC classification method can be integrated into the existing hazard mapping process to enable a full automation of the process and provide a more accurate basis for disaster mapping. This also requires a deeper analysis of the complexity of the relationship between different LULC classes and rainfall-runoff, and the impact of topography and geomorphology on water flow. This is intended to make the results of the LULC classification more effective in practical applications.

CRediT authorship contribution statement

Dennis Sprute: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Hanne Hendrickx:** Writing – review & editing, Writing – original draft, Resources. **Pedro Zamboni:** Writing – review & editing, Writing – original draft. **Muhtasimul Islam Rushdi:** Writing – review & editing, Writing – original draft. **Florian Brodrecht:** Writing – review & editing, Funding acquisition. **Anette Eltner:** Writing – review & editing, Funding acquisition. **Holger Flatt:** Writing – review & editing, Funding acquisition.

Ethical statement

We confirm that this manuscript is original and has not been published elsewhere, nor it is currently under consideration for publication in any other journal. The manuscript has been prepared in accordance with the journal's guidelines for manuscript submission. Furthermore, we declare that there are no conflicts of interest associated with this work.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used FhGenie (Weber et al., 2024) in order to improve the readability and language of parts of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Funding

This work was financially supported by the German Federal Ministry for Economic Affairs and Climate Action (Funding numbers: KK5145103PR3, KK5549901PR3 and KK5548201PR3).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Example images of the classification system

See [Tables A.9–A.12](#).

Table A.9
Example images of the *Street* macro-class (areas enclosed in cyan).

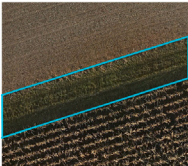
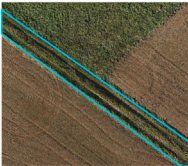
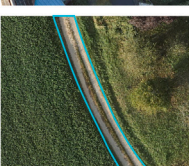
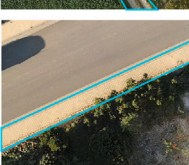
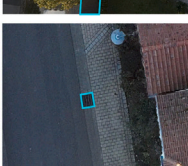
Macro-class	Class	Examples
Street	Street_field	
		
		
	Street_paved	
		
		
	Street_semipaved	
		
		
	Street_unpaved	
		
		
Sidewalk	Inlet	
		
		
Manhole	Manhole	
		
		

Table A.10
Example images of the *Rural* and *Rivers & trenches* macro-classes (areas enclosed in cyan).

Macro-class	Class	Examples
Rural	Farmland	
	Greenland	
	Woodland	
Rivers & trenches	Pipe	
	Bridge	
	Water	

Table A.11
Example images of the *Residential* macro-class (areas enclosed in cyan).

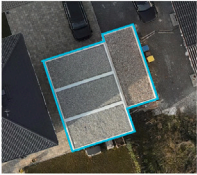
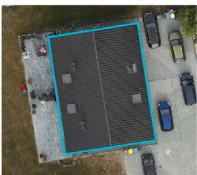
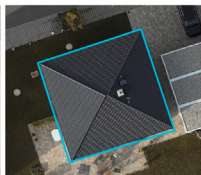
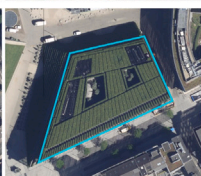
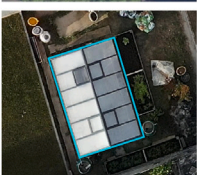
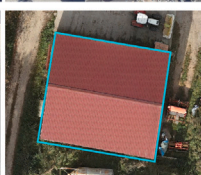
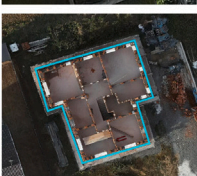
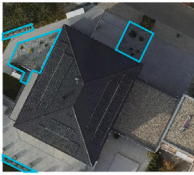
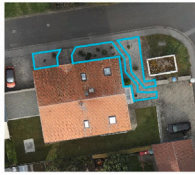
Macro-class	Class	Examples
Residential	Roof_flat	  
	Roof_tiled	  
	Roof_green	  
	Roof_tin	  
	Building_site	  

Table A.12
Example images of the *Residential* macro-class (areas enclosed in cyan).

Macro-class	Class	Examples
Residential	Garden_green	  
	Garden_stone	  
	Sealed_surface	  
	PV_system	  

Appendix B. Detailed evaluation results

See Figs. B.8 and B.9 and Table B.13.

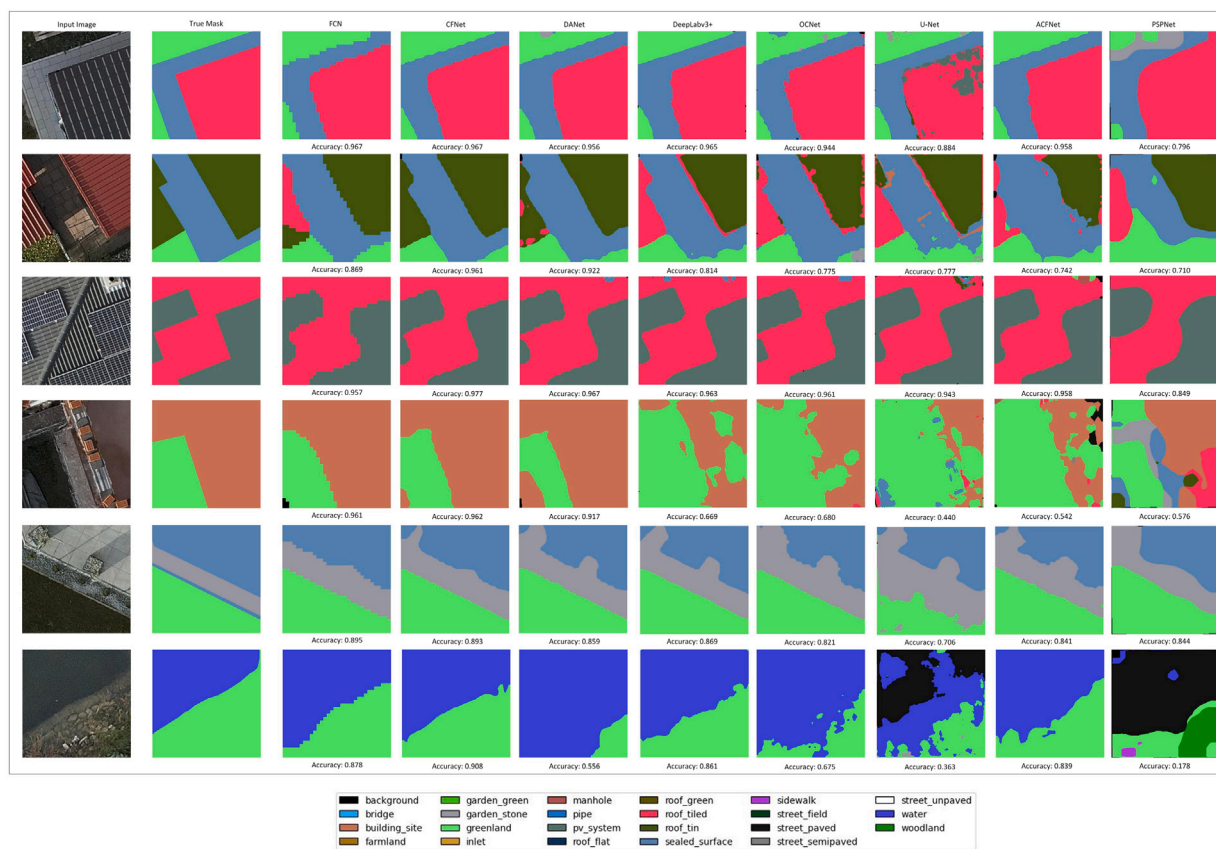


Fig. B.8. Examples of semantic segmentation architectures with EfficientNetB4 backbone and custom class weights during training on images of the test dataset. Columns indicate the input image (left), ground truth mask (second column) and predicted masks of models (following columns), while rows show samples.

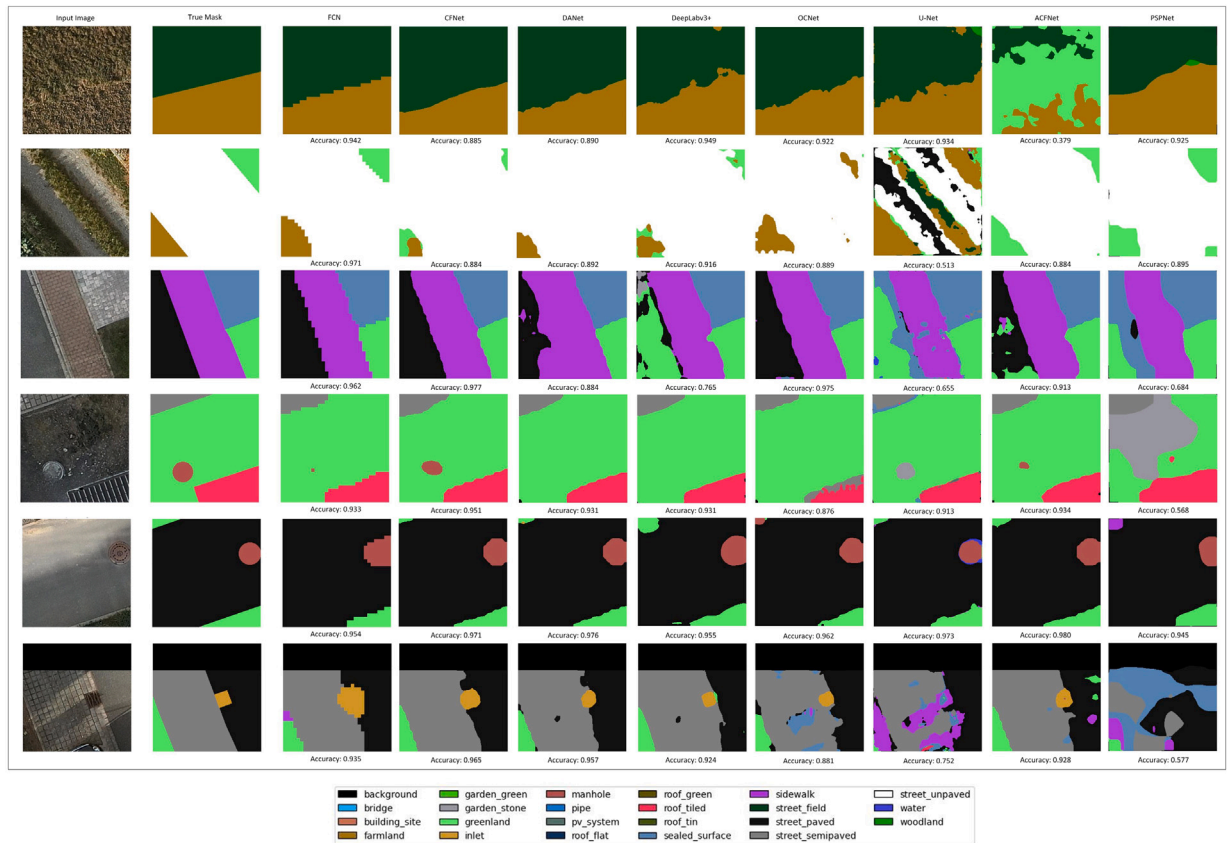


Fig. B.9. Examples of semantic segmentation architectures with EfficientNetB4 backbone and custom class weights during training on images of the test dataset. Columns indicate the input image (left), ground truth mask (second column) and predicted masks of models (following columns), while rows show samples.

Table B.13

Performance of different models ordered by their accuracy in descending order. Columns containing IoU values are ordered by their mean value in descending order from left to right.

Architecture	Backbone	Accuracy	Mean IoU	IoU																		
				pr_system	roof_tiled	street_paved	background	farmland	street_unpaved	greenland	woodland	street_sampraved	sealed_surface	roof_flat	street_field	roof_tin	building_site	sidewalk	garden_zone	manhole	water	inlet
FCN	EfficientNetB4	0.927	0.734	0.918	0.893	0.923	0.932	0.872	0.886	0.821	0.856	0.765	0.766	0.793	0.763	0.684	0.914	0.762	0.495	0.530	0.366	0.000
FCN	Xception	0.922	0.721	0.932	0.904	0.933	0.883	0.897	0.886	0.802	0.804	0.756	0.752	0.798	0.773	0.700	0.936	0.688	0.375	0.521	0.356	0.000
CFNet	EfficientNetB4	0.906	0.716	0.941	0.910	0.926	0.899	0.859	0.864	0.765	0.765	0.766	0.764	0.759	0.699	0.649	0.885	0.675	0.470	0.537	0.473	0.000
CFNet	Xception	0.906	0.695	0.919	0.892	0.926	0.890	0.843	0.875	0.776	0.764	0.749	0.733	0.714	0.735	0.619	0.901	0.647	0.351	0.527	0.340	0.000
FCN	ResNet101	0.904	0.656	0.898	0.879	0.906	0.882	0.878	0.883	0.773	0.793	0.718	0.695	0.721	0.710	0.575	0.866	0.590	0.310	0.217	0.170	0.000
FCN	MobileNetV2	0.891	0.639	0.915	0.887	0.895	0.874	0.808	0.851	0.742	0.747	0.694	0.702	0.744	0.602	0.559	0.811	0.690	0.396	0.197	0.019	0.000
DANet	EfficientNetB4	0.890	0.664	0.901	0.883	0.880	0.848	0.833	0.762	0.749	0.740	0.691	0.708	0.681	0.693	0.665	0.809	0.556	0.489	0.500	0.210	0.011
DANet	Xception	0.888	0.628	0.898	0.859	0.883	0.856	0.816	0.795	0.749	0.761	0.706	0.673	0.665	0.699	0.614	0.712	0.491	0.221	0.391	0.133	0.001
DANet	ResNet101	0.879	0.601	0.884	0.840	0.874	0.858	0.806	0.766	0.742	0.772	0.633	0.636	0.639	0.658	0.534	0.680	0.453	0.201	0.321	0.117	0.000
CFNet	MobileNetV2	0.874	0.648	0.908	0.843	0.842	0.851	0.819	0.831	0.721	0.703	0.700	0.675	0.631	0.645	0.526	0.659	0.565	0.480	0.374	0.544	0.000
OCNet	EfficientNetB4	0.869	0.587	0.849	0.826	0.849	0.816	0.820	0.773	0.677	0.732	0.613	0.621	0.583	0.686	0.530	0.712	0.433	0.237	0.294	0.096	0.000
CFNet	ResNet101	0.866	0.591	0.855	0.818	0.830	0.858	0.785	0.856	0.722	0.749	0.642	0.604	0.574	0.583	0.596	0.718	0.468	0.416	0.114	0.044	0.000
DeepLabv3+	EfficientNetB4	0.857	0.591	0.800	0.850	0.831	0.901	0.805	0.798	0.673	0.686	0.629	0.644	0.604	0.484	0.556	0.656	0.405	0.335	0.464	0.101	0.000
DeepLabv3+	MobileNetV2	0.849	0.537	0.818	0.791	0.851	0.843	0.769	0.719	0.677	0.672	0.612	0.625	0.568	0.515	0.479	0.507	0.428	0.284	0.049	0.000	0.000
DANet	MobileNetV2	0.842	0.530	0.799	0.823	0.795	0.768	0.769	0.631	0.657	0.691	0.653	0.617	0.554	0.579	0.486	0.561	0.354	0.147	0.164	0.021	0.000
U-Net	EfficientNetB4	0.838	0.479	0.705	0.781	0.811	0.809	0.804	0.676	0.686	0.708	0.590	0.549	0.499	0.577	0.409	0.170	0.253	0.074	0.000	0.000	0.000
OCNet	Xception	0.837	0.535	0.821	0.818	0.818	0.812	0.757	0.671	0.599	0.654	0.674	0.595	0.576	0.555	0.601	0.497	0.373	0.128	0.214	0.000	0.000
OCNet	ResNet101	0.826	0.498	0.785	0.788	0.816	0.836	0.762	0.699	0.585	0.640	0.628	0.554	0.542	0.546	0.565	0.271	0.267	0.121	0.041	0.023	0.000
U-Net	MobileNetV2	0.826	0.478	0.799	0.809	0.809	0.853	0.744	0.553	0.649	0.660	0.559	0.588	0.433	0.575	0.408	0.335	0.270	0.043	0.000	0.003	0.000
U-Net	ResNet101	0.820	0.476	0.759	0.798	0.806	0.869	0.757	0.614	0.632	0.671	0.571	0.498	0.455	0.583	0.401	0.302	0.265	0.066	0.000	0.000	0.000
ACFNet	EfficientNetB4	0.808	0.518	0.827	0.846	0.694	0.822	0.790	0.837	0.581	0.360	0.574	0.626	0.561	0.535	0.520	0.239	0.491	0.213	0.319	0.000	0.000
OCNet	MobileNetV2	0.798	0.447	0.782	0.769	0.776	0.777	0.697	0.415	0.556	0.578	0.551	0.491	0.492	0.469	0.446	0.358	0.274	0.050	0.013	0.000	0.000
DeepLabv3+	ResNet101	0.794	0.483	0.801	0.806	0.842	0.840	0.661	0.795	0.561	0.432	0.619	0.573	0.566	0.231	0.470	0.416	0.377	0.150	0.030	0.017	0.000
ACFNet	ResNet101	0.777	0.468	0.862	0.833	0.832	0.746	0.649	0.844	0.541	0.271	0.546	0.546	0.623	0.311	0.606	0.019	0.400	0.195	0.058	0.000	0.000
PSPNet	EfficientNetB4	0.764	0.445	0.684	0.719	0.689	0.371	0.618	0.763	0.553	0.581	0.462	0.440	0.546	0.527	0.460	0.464	0.310	0.194	0.050	0.018	0.000
ACFNet	Xception	0.763	0.475	0.803	0.844	0.847	0.798	0.643	0.734	0.506	0.068	0.599	0.652	0.628	0.184	0.552	0.281	0.485	0.267	0.090	0.047	0.000
ACFNet	MobileNetV2	0.757	0.475	0.817	0.788	0.799	0.806	0.664	0.794	0.515	0.087	0.622	0.601	0.612	0.209	0.548	0.299	0.356	0.275	0.240	0.000	0.000
PSPNet	ResNet101	0.671	0.282	0.611	0.658	0.614	0.282	0.510	0.467	0.450	0.408	0.042	0.278	0.346	0.169	0.367	0.002	0.082	0.072	0.000	0.000	0.000
PSPNet	MobileNetV2	0.658	0.288	0.610	0.673	0.201	0.327	0.592	0.474	0.458	0.382	0.380	0.320	0.034	0.295	0.468	0.079	0.096	0.078	0.000	0.000	0.000

Data availability

The authors do not have permission to share data.

References

- Baek, K., Bang, D., Shim, H., 2021. GridMix: Strong regularization through local context mapping. *Pattern Recognit.* 109, 1–10.
- Cai, W., Jin, K., Hou, J., Guo, C., Wu, L., Yang, W., 2023. VDD: Varied drone dataset for semantic segmentation. <https://arxiv.org/abs/2305.13608>.
- Chen, L.C., Papandreou, G., Schroff, F., Adam, H., 2017. Rethinking atrous convolution for semantic image segmentation. <https://arxiv.org/abs/1706.05587>.
- Chen, L.C., Zhu, Y., Papandreou, G., Schroff, F., Adam, H., 2018. Encoder–decoder with atrous separable convolution for semantic image segmentation. In: *Computer Vision – ECCV 2018*. pp. 833–851.
- Chollet, F., 2017. Xception: Deep learning with depthwise separable convolutions. In: *IEEE Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 1800–1807.
- Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., Schiele, B., 2016. The cityscapes dataset for semantic urban scene understanding. In: *IEEE Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 3213–3223.
- Cubuk, E.D., Zoph, B., Mané, D., Vasudevan, V., Le, Q.V., 2019. AutoAugment: Learning augmentation strategies from data. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 113–123.
- de Moel, H., van Alphen, J., Aerts, J.C.J.H., 2009. Flood maps in europe – methods, availability and use. *Nat. Hazards Earth Syst. Sci.* 9, 289–301.
- Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L., 2009. ImageNet: A large-scale hierarchical image database. In: *IEEE Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 248–255.
- Dou, P., Shen, H., Huang, C., Li, Z., Mao, Y., Li, X., 2024. Large-scale land use/land cover extraction from landsat imagery using feature relationships matrix based deep-shallow learning. *Int. J. Appl. Earth Obs. Geoinf.* 129, 1–17.
- Eltner, A., Hoffmeister, D., Karrasch, P., Klingbeil, L., Stöcker, C., Rovere, A., Kaiser, A., 2022. UAVs for the Environmental Sciences. wbg Academic.
- Fu, J., Liu, J., Tian, H., Li, Y., Bao, Y., Fang, Z., Lu, H., 2019. Dual attention network for scene segmentation. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 3141–3149.
- Garcia-Garcia, A., Orts-Escolano, S., Oprea, S., Villena-Martinez, V., Martinez-Gonzalez, P., Garcia-Rodriguez, J., 2018. A survey on deep learning techniques for image and video semantic segmentation. *Appl. Soft Comput.* 70, 41–65.
- GDV, DWD, 2019. Forschungsprojekt Starkregen – Fachbericht (in German). Technical Report, GDV.
- Giang, T.L., Dang, K.B., Toan Le, Q., Nguyen, V.G., Tong, S.S., Pham, V.M., 2020. U-Net convolutional networks for mining land cover classification based on high-resolution UAV imagery. *IEEE Access* 8, 186257–186273.
- Girisha, S., Pai, M.M.M., Verma, U., Pai, R.M., 2019. Performance analysis of semantic segmentation algorithms for finely annotated new UAV aerial video dataset (ManipalUAVid). *IEEE Access* 7, 136239–136253.
- Gonzalez, R.C., Woods, R.E., 2017. *Digital Image Processing*, fourth ed. Pearson.
- Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y., 2014. Generative adversarial nets. In: *Advances in Neural Information Processing Systems*. pp. 1–9.
- He, D., Shi, Q., Xue, J., Atkinson, P.M., Liu, X., 2023. Very fine spatial resolution urban land cover mapping using an explicable sub-pixel mapping network based on learnable spatial correlation. *Remote Sens. Environ.* 299, 1–20.
- He, K., Zhang, X., Ren, S., Sun, J., 2016. Deep residual learning for image recognition. In: *IEEE Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 770–778.
- Hendrickx, H., Vivero, S., Cock, L.D., Wit, B.D., Maeyer, P.D., Lambiel, C., Delaloye, R., Nyssen, J., Frankl, A., 2019. The reproducibility of SfM algorithms to produce detailed digital surface models: The example of photoscan applied to a high-alpine rock glacier. *Remote. Sens. Lett.* 10, 11–20.
- Jia, P., Chen, C., Zhang, D., Sang, Y., Zhang, L., 2024. Semantic segmentation of deep learning remote sensing images based on band combination principle: Application in urban planning and land use. *Comput. Commun.* 217, 97–106.
- Jongman, B., Hochrainer-Stigler, S., Feyen, L., Aerts, J.C.J.H., Mechler, R., Botzen, W.J.W., Bouwer, L.M., Pflug, G., Rojas, R., Ward, P.J., 2014. Increasing stress on disaster-risk finance due to large floods. *Nat. Clim. Chang.* 4, 264–268.
- Kimura, Y., Hirabayashi, Y., Kita, Y., Zhou, X., Yamazaki, D., 2023. Methodology for constructing a flood-hazard map for a future climate. *Hydrol. Earth Syst. Sci.* 27, 1627–1644.
- Kingma, D.P., Ba, J., 2015. Adam: A method for stochastic optimization. In: *International Conference on Learning Representations*. ICLR, pp. 1–15.
- Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y., Dollár, P., Girshick, R., 2023. Segment anything. In: *IEEE/CVF International Conference on Computer Vision*. ICCV, pp. 3992–4003.
- Leal, M., Boavida-Portugal, I., Frago, M., Ramos, C., 2019. How much does an extreme rainfall event cost? material damage and relationships between insurance, rainfall, land cover and urban flooding. *Hydrol. Sci. J.* 64, 673–689.
- Li, Z., Chen, B., Wu, S., Su, M., Chen, J.M., Xu, B., 2024. Deep learning for urban land use category classification: A review and experimental assessment. *Remote Sens. Environ.* 311, 114290.
- Long, J., Shelhamer, E., Darrell, T., 2015. Fully convolutional networks for semantic segmentation. In: *IEEE Conference on Computer Vision and Pattern Recognition*. CVPR, pp. 3431–3440.
- LUBW, 2016. Leitfaden Kommunales Starkregenrisikomanagement in Baden-Württemberg (in German). Technical Report, LUBW Landesanstalt für Umwelt, Messungen und Naturschutz Baden-Württemberg.
- Lyu, Y., Vosselman, G., Xia, G.S., Yilmaz, A., Yang, M.Y., 2020. UAVid: A semantic segmentation dataset for UAV imagery. *ISPRS J. Photogramm. Remote Sens.* 165, 108–119.
- Moghim, S., Gharehtoragh, M.A., Safaie, A., 2023. Performance of the flood models in different topographies. *J. Hydrol.* 620, 1–11.
- Nigam, I., Huang, C., Ramanan, D., 2018. Ensemble knowledge transfer for semantic segmentation. In: *IEEE Winter Conference on Applications of Computer Vision*. WACV, pp. 1499–1508.
- Osco, L.P., Marcato Junior, J., Marques Ramos, A.P., de Castro Jorge, L.A., Fatholahi, S.N., de Andrade Silva, J., Matsubara, E.T., Pistori, H., Gonçalves, W.N., Li, J., 2021. A review on deep learning in UAV remote sensing. *Int. J. Appl. Earth Obs. Geoinf.* 102, 1–21.
- Ponce, V.M., Hawkins, R.H., 1996. Runoff curve number: Has it reached maturity? *J. Hydrol. Eng.* 1, 11–19.
- Rahmounfar, M., Chowdhury, T., Sarkar, A., Varshney, D., Yari, M., Murphy, R.R., 2021. FloodNet: A high resolution aerial imagery dataset for post flood scene understanding. *IEEE Access* 9, 89644–89654.
- Rallison, R.E., 1980. Origin and evolution of the SCS runoff equation. In: *Proceeding of the Symposium on Watershed Management*.
- Rangari, V.A., Umamahesh, N.V., Bhatt, C.M., 2019. Assessment of inundation risk in urban floods using HEC RAS 2D. *Model. Earth Syst. Environ.* 5, 1839–1851.
- Ren, T., Liu, S., Zeng, A., Lin, J., Li, K., Cao, H., Chen, J., Huang, X., Chen, Y., Yan, F., Zeng, Z., Zhang, H., Li, F., Yang, J., Li, H., Jiang, Q., Zhang, L., 2024. Grounded SAM: Assembling open-world models for diverse visual tasks. <https://arxiv.org/abs/2401.14159>.
- Ren, P., Xiao, Y., Chang, X., Huang, P.Y., Li, Z., Gupta, B.B., Chen, X., Wang, X., 2021. A survey of deep active learning. *ACM Comput. Surv.* 54.

- Ronneberger, O., Fischer, P., Brox, T., 2015. U-Net: Convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention. MICCAI, Springer International Publishing, pp. 234–241.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.C., 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, pp. 4510–4520.
- Tan, M., Le, Q., 2019. EfficientNet: Rethinking model scaling for convolutional neural networks. In: International Conference on Machine Learning. ICML, pp. 6105–6114.
- Tang, Z., Peng, X., Li, T., Zhu, Y., Metaxas, D., 2019. AdaTransform: Adaptive data transformation. In: IEEE/CVF International Conference on Computer Vision. ICCV, pp. 2998–3006.
- Thrysoe, C., Balström, T., Borup, M., Löwe, R., Jamali, B., Arnbjerg-Nielsen, K., 2021. FloodStroem: A fast dynamic GIS-based urban flood and damage model. *J. Hydrol.* 600, 126521.
- Wagner, B., Egerer, M., 2022. Application of UAV remote sensing and machine learning to model and map land use in urban gardens. *J. Urban Ecol.* 8, 1–12.
- Wagner, F., Eltner, A., Maas, H.G., 2023. River water segmentation in surveillance camera images: A comparative study of offline and online augmentation using 32 CNNs. *Int. J. Appl. Earth Obs. Geoinf.* 119, 1–22.
- Wang, L., Li, R., Zhang, C., Fang, S., Duan, C., Meng, X., Atkinson, P.M., 2022. UNetFormer: A UNet-like transformer for efficient semantic segmentation of remote sensing urban scene imagery. *ISPRS J. Photogramm. Remote Sens.* 190, 196–214.
- Wang, S., Wang, Y., Zhang, L., Xu, M., Yao, X., Wang, X., 2024. Associated impact mechanism of heavy rain and floods in the middle and lower reaches of the yangtze river basin based on ocean-atmosphere anomaly patterns. *Sci. Total Environ.* 946, 1–14.
- Wasko, C., Nathan, R., Stein, L., O'Shea, D., 2021. Evidence of shorter more extreme rainfalls and increased flood variability under climate change. *J. Hydrol.* 603, 1–12.
- Weber, I., Linka, H., Mertens, D., Muryshkin, T., Opgenoorth, H., Langer, S., 2024. FhGenie: A custom, confidentiality-preserving chat AI for corporate and scientific use. <https://arxiv.org/abs/2403.00039>.
- Xu, M., Yoon, S., Fuentes, A., Park, D.S., 2023. A comprehensive survey of image augmentation techniques for deep learning. *Pattern Recognit.* 137, 1–12.
- Yalcin, E., 2020. Assessing the impact of topography and land cover data resolutions on two-dimensional hec-ras hydrodynamic model simulations for urban flood hazard analysis. *Nat. Hazards* 101, 995–1017.
- Yi, S., Liu, X., Li, J., Chen, L., 2023. UAVformer: A composite transformer network for urban scene segmentation of UAV images. *Pattern Recognit.* 133, 1–19.
- Yuan, Y., Huang, L., Guo, J., Zhang, C., Chen, X., Wang, J., 2021. OCNet: Object context for semantic segmentation. *Int. J. Comput. Vis.* 129, 2375–2398.
- Yun, S., Han, D., Chun, S., Oh, S.J., Yoo, Y., Choe, J., 2019. CutMix: Regularization strategy to train strong classifiers with localizable features. In: IEEE/CVF International Conference on Computer Vision. ICCV, pp. 6022–6031.
- Zamboni, P.A.P., Hendrickx, H., Sprute, D., Flatt, H., Rushdi, M.I., Brodrecht, F., Eltner, A., 2024. Synergistic image and point cloud processing of UAV data for urban flood modeling: Point cloud smart thinning and curb mapping. *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.* XLVIII-2/W8-2024, 483–490.
- Zeng, Z., Tang, G., Hong, Y., Zeng, C., Yang, Y., 2017. Development of an NRCS curve number global dataset using the latest geospatial remote sensing data for worldwide hydrologic applications. *Remote. Sens. Lett.* 8, 528–536.
- Zhang, F., Chen, Y., Li, Z., Hong, Z., Liu, J., Ma, F., Han, J., Ding, E., 2019b. ACFNet: Attentional class feature network for semantic segmentation. In: IEEE/CVF International Conference on Computer Vision. ICCV, pp. 6797–6806.
- Zhang, Y., Huang, X., Ma, J., Li, Z., Zhaochuan Luo, Y.X., Qin, Y., Luo, T., Li, Y., Liu, S., Guo, Y., Zhang, L., 2023. Recognize anything: A strong image tagging model. <https://arxiv.org/abs/2306.03514>.
- Zhang, C., Sargent, I., Pan, X., Li, H., Gardiner, A., Hare, J., Atkinson, P.M., 2019a. Joint deep learning for land cover and land use classification. *Remote Sens. Environ.* 221, 173–187.
- Zhang, H., Zhang, H., Wang, C., Xie, J., 2019c. Co-occurrent features in semantic segmentation. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR, pp. 548–557.
- Zhao, H., Shi, J., Qi, X., Wang, X., Jia, J., 2017. Pyramid scene parsing network. In: IEEE Conference on Computer Vision and Pattern Recognition. CVPR, pp. 6230–6239.
- Zhou, B., Zhao, H., Puig, X., Fidler, S., Barriuso, A., Torralba, A., 2017. Scene parsing through ADE20K dataset. In: IEEE Conference on Computer Vision and Pattern Recognition. CVPR, pp. 5122–5130.